

# Caspian: A Carbon-aware Workload Scheduler in Multi-Cluster Kubernetes Environments

Tayebeh Bahreini · Asser N. Tantawi · Olivier Tardieu

Wayne State University · IBM T.J. Watson Research Center (x2)

IEEE MASCOTS 2024 | DOI: 10.1109/MASCOTS64422.2024.10786568

📖 읽는 법 | 원문 → 해석(한국어) → 설명(초등학생도 OK) 순서로 모든 문장을 빠짐없이 정리

|    |                                                                                                                          |
|----|--------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>Index Terms — cloud sustainability, green computing, Kubernetes, multi-cluster management, placement, scheduling.</i> |
| 해석 | 키워드 — 클라우드 지속가능성, 친환경 컴퓨팅, 쿠버네티스, 멀티클러스터 관리, 배치, 스케줄링.                                                                   |
| 설명 | 이 논문이 다루는 주제들을 한 단어씩 정리한 것이다. 학술 검색할 때 이 단어들로 찾을 수 있다.                                                                   |

## Abstract 초록

|    |                                                                                                                                                                                                                     |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>The surge in demand for computing resources in data centers coupled with the rise of environmental concerns has motivated cloud providers to reduce carbon emission due to computational energy consumption.</i> |
| 해석 | 데이터센터의 컴퓨팅 자원 수요가 폭발적으로 늘어나고 환경 문제에 대한 우려도 함께 커지면서, 클라우드 사업자들은 컴퓨팅 에너지 소비로 인한 탄소 배출을 줄이려는 동기를 갖게 됐다.                                                                                                                |
| 설명 | 쉽게 말하면: 요즘 AI, 유튜브, 게임 등 인터넷 서비스가 폭발적으로 늘면서, 그것들을 돌리는 데이터센터가 전기를 엄청나게 먹고 있다. 전기를 많이 쓸수록 이산화탄소(CO <sub>2</sub> )가 더 많이 나온다. 그래서 "어떻게 하면 같은 일을 하면서 CO <sub>2</sub> 를 덜 낼 수 있을까?"라는 질문이 생긴 것이다.                        |

|    |                                                                                                                                                                                                                                      |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>An opportunity lies in the fluctuating availability of renewable energy over time and the variability of power sources over grid regions, leading to variations in space and time in carbon intensity.</i>                        |
| 해석 | 시간에 따라 재생에너지 가용량이 변동하고, 지역마다 전력원이 다르다는 점에서 기회가 생긴다. 이는 탄소집약도(carbon intensity)가 시간과 공간에 따라 달라진다는 것을 의미한다.                                                                                                                            |
| 설명 | 탄소집약도란? 전기 1kWh를 만들 때 나오는 CO <sub>2</sub> 의 양(g)이다. 태양광·수력·풍력 같은 재생에너지 비중이 높으면 낮고, 석탄·가스 화력발전이 많으면 높다.<br>쉽게 말하면: 캐나다는 수력발전이 많아서 "깨끗한 전기"를 쓴다. 독일은 가스발전이 많아서 "더러운 전기"를 쓴다. 같은 계산을 캐나다에서 하면 CO <sub>2</sub> 가 훨씬 적게 나온다! 이것 이용자는 것이다. |

|    |                                                                                                                                                                                                                                                                                     |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>Exploiting such variations, this paper introduces Caspian, a carbon-aware workload scheduler in multi-cluster Kubernetes environments, which aims at reducing the Carbon Footprint (CFP) due to executing workloads, while satisfying Quality of Service (QoS) requirements.</i> |
| 해석 | 이러한 변동성을 활용하여, 이 논문은 멀티클러스터 쿠버네티스 환경을 위한 탄소인지 워크로드 스케줄러 Caspian을 소개한다. Caspian은 QoS 요구사항을 만족하면서 워크로드 실행에 따른 탄소발자국(CFP)을 줄이는 것을 목표로 한다.                                                                                                                                              |
| 설명 | 탄소발자국(CFP): 내가 한 행동(여기서는 컴퓨터 작업)으로 발생한 CO <sub>2</sub> 의 총량.<br>QoS(서비스 품질): 쉽게 말하면 "제시간에 작업을 끝내는 것". 탄소를 줄인다고 작업을 너무 오래 미루면 안 된다.<br>Caspian이 하는 일: "지금 이 계산 작업을 어느 나라 서버에서, 언제 돌릴까?"를 결정하는 스케줄러다. 최대한 깨끗한 전기를 쓰는 곳에서, 마감 기한 내에 돌리는 것이 목표.                                         |

|    |                                                                                                                                                                                                                |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>Caspian cooperates with a multi-cluster management platform to apply scheduling and placement decisions over distributed clusters. We present efficient optimization algorithms to achieve these goals.</i> |
| 해석 | Caspian은 멀티클러스터 관리 플랫폼과 협력하여 분산된 클러스터 전반에 스케줄링·배치 결정을 적용한다. 우리는 이 목표를 달성하기 위한 효율적인 최적화 알고리즘을 제시한다.                                                                                                             |
| 설명 | 멀티클러스터: 여러 지역에 흩어진 서버 묶음들. 예: 캐나다 서버 묶음, 일본 서버 묶음, 독일 서버 묶음.                                                                                                                                                   |

|    |                                                                                                                                                                                                                                                                                                                                                      |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | <b>Caspian의 역할:</b> Caspian은 직접 작업을 실행하지 않는다. "이 작업은 캐나다 서버에서 오후 3시에 시작해"라고 지시만 내린다. 실제 실행은 MCAD라는 별도 플랫폼이 한다. <b>우리의 플랫폼도 이런 구성이면 좋을듯</b>                                                                                                                                                                                                           |
| 원문 | <i>Further, we describe an implementation of Caspian, integrated with Multi Cluster App Dispatcher (MCAD), a multi-cluster management platform which handles queuing and dispatching of workloads over multiple clusters.</i>                                                                                                                        |
| 해석 | 더 나아가, 우리는 여러 클러스터에 걸쳐 워크로드의 큐잉과 디스패치를 처리하는 멀티클러스터 관리 플랫폼인 MCAD와 통합된 Caspian의 구현을 설명한다.                                                                                                                                                                                                                                                              |
| 설명 | <b>큐잉(queuing):</b> 작업들을 줄 세워 기다리게 하는 것. 마치 맛집 앞 대기줄과 같다.<br><b>디스패치(dispatch):</b> 대기 중인 작업을 실제로 서버에 보내서 실행시키는 것.<br><b>MCAD:</b> IBM/Red Hat이 만든 오픈소스 소프트웨어. Caspian이 "캐나다로 보내"라고 하면 MCAD가 실제로 보내준다. 소스코드: <a href="https://github.com/project-codeflare/mcad">github.com/project-codeflare/mcad</a> <b>MCAD???</b><br><b>해당 오픈소스 소프트웨어 하면 좋을듯</b> |
| 원문 | <i>Our experimental results show that Caspian effectively reduces CFP with reasonable QoS, compared to a baseline scheduler which only satisfies the QoS of workloads. Specifically, Caspian reduces CFP by about 33%, with about 98% of workloads completing at an average fraction of 0.6 of their deadline.</i>                                   |
| 해석 | <b>실험 결과, QoS만 만족시키는 베이스라인 스케줄러와 비교했을 때 Caspian은 합리적인 QoS를 유지하면서 CFP를 효과적으로 줄인다. 구체적으로 Caspian은 CFP를 약 33% 줄이며, 워크로드의 약 98%가 마감기한의 평균 0.6 지점에서 완료된다.</b>                                                                                                                                                                                             |
| 설명 | <b>33% 탄소 절감:</b> 기존 방식으로 하면 탄소를 100만큼 냈다면, Caspian을 쓰면 67만큼만 낸다.<br><b>98% 마감 준수:</b> 100개 작업 중 98개는 기한 내에 끝낸다. <b>여기는 소프트웨어조건 때문에 해당 지표를 쓰는듯</b><br><b>평균 0.6 지점:</b> 마감까지 남은 시간의 60% 지점에서 끝난다는 뜻. 예를 들어 마감이 10시간 뒤라면 평균 6시간 만에 완료된다. 여유 있게 끝낸다는 의미.<br><b>⚠ 이 수치는 탄소 가중치 <math>\omega_t=0.5</math>로 설정했을 때다. 이 값을 바꾸면 결과도 달라진다.</b>               |

-----

## I. Introduction (서론)

|    |                                                                                                                                                                                                                                                                                                                              |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>Over the past few years, due to the surge in using cloud computing for executing compute intensive jobs, such as artificial intelligence and machine learning (AI/ML) workloads, data centers have contributed to the production of a significant CFP compared to some other industries such as the airline industry.</i> |
| 해석 | 최근 몇 년간, AI/ML 워크로드와 같은 연산 집약적 작업에 클라우드 컴퓨팅 사용이 급증하면서, 데이터센터는 항공 산업 등 일부 다른 산업에 비해 상당한 탄소발자국을 생산해왔다.                                                                                                                                                                                                                         |
| 설명 | ChatGPT 같은 AI 서비스를 한 번 학습시키는 데 엄청난 전기가 필요하다. 이런 AI 작업들이 폭발적으로 늘면서 데이터센터의 탄소 배출이 항공업계만큼 심각해지고 있다.                                                                                                                                                                                                                             |

|    |                                                                                                                                                                                                                                                                          |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>Data centers consume approximately 1–2% of global electricity production [13], resulting in an estimated 2.5–3.7% of global carbon emission [7]. This trend will most likely continue in the next decade, leading to a concerning contribution to global warming.</i> |
| 해석 | 데이터센터는 전 세계 전력 생산량의 약 1~2%를 소비하며[13], 이는 전 세계 탄소 배출량의 약 2.5~3.7%에 해당하는 것으로 추산된다[7]. 이 추세는 향후 10년간 계속될 가능성이 높아, 지구온난화에 우려스러운 수준으로 기여하게 될 것이다.                                                                                                                             |
| 설명 | 전기 소비가 1~2%인데 탄소 배출이 2.5~3.7%인 이유: 데이터센터가 탄소를 많이 내는 화력발전 전기를 특히 많이 쓰기 때문이다.<br>이 추세대로 가면 10년 후엔 훨씬 심각해진다. 지금 해결 방법을 찾아야 한다.                                                                                                                                              |

|    |                                                                                                                                                                                                                                                                             |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>Many technology companies have invested in renewable energy to lower the energy cost and the environmental impact of their services [11], [12]. These companies either generate their own renewable energy onsite or directly supply it from renewable power plants.</i> |
| 해석 | 많은 기술 기업들이 서비스의 에너지 비용과 환경 영향을 낮추기 위해 재생에너지에 투자했다[11],[12]. 이 기업들은 자체 재생에너지를 현장에서 생산하거나 재생에너지 발전소에서 직접 공급받는다.                                                                                                                                                               |
| 설명 | 구글, 마이크로소프트 같은 회사들은 이미 태양광 패널을 데이터센터 지붕에 설치하거나, 풍력발전소에서 전기를 사오는 계약(PPA)을 맺고 있다.                                                                                                                                                                                             |

|    |                                                                                                                                                                              |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>However, achieving carbon-free computing still remains challenging due to the uncertain and variable availability of renewable energy sources such as solar and wind.</i> |
| 해석 | 그러나 태양광·풍력 같은 재생에너지 원의 불확실하고 가변적인 가용성으로 인해, 탄소없는 컴퓨팅을 달성하는 것은 여전히 어려운 과제로 남아 있다.                                                                                             |
| 설명 | 문제는 태양광은 구름 끼면 안 되고, 풍력은 바람 없으면 안 된다. 재생에너지는 날씨에 따라 들쭉날쭉해서 항상 충분하지 않다.                                                                                                       |

|    |                                                                                                                                                                                                               |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>One potential solution to tackle this variability is to store energy. However, there are concerns about using batteries for renewable energy storage such as cost and their impact on the environment.</i> |
| 해석 | 이러한 변동성에 대응하는 한 가지 방법은 에너지를 저장하는 것이다. 그러나 배터리를 재생에너지 저장에 사용하는 데는 비용이나 환경에 미치는 영향 같은 우려가 있다.                                                                                                                   |
| 설명 | 배터리에 전기를 저장해뒀다가 쓰면 되지 않냐? — 맞다. 하지만 대형 배터리는 비싸고, 만들고 버릴 때 오히려 환경 오염이 생긴다. 완벽한 해답이 아니다.                                                                                                                        |

|    |                                                                                                                                                                                                                                                                                                                                  |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>Another approach is to dynamically manage the execution of workloads to match the low-carbon energy supply. This approach requires revisiting the design of resource management platforms in data centers as they mainly focus on optimizing the trade off between performance, utilization, and energy consumption [14].</i> |
| 해석 | 또 다른 접근법은 저탄소 에너지 공급에 맞춰 워크로드 실행을 동적으로 관리하는 것이다. 이 접근법은 데이터센터 자원관리 플랫폼의 설계를 재검토할 것을 요구한다. 기존 플랫폼은 주로 성능, 사용률, 에너지 소비 간의 트레이드오프를 최적화하는 데 초점을 맞추고 있기 때문이다[14].                                                                                                                                                                     |
| 설명 | 이 논문의 핵심 아이디어: 전기를 저장하는 게 아니라, 계산 작업을 깨끗한 전기가 있는 시간/장소로 옮기자!<br>예시: "지금 독일이 석탄 전기를 쓰고 있으니까 이 작업을 2시간 후로 미뤄서 캐나다에서 돌리자."                                                                                                                                                                                                          |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | 기존 스케줄러는 속도·효율만 생각했지 탄소는 전혀 안 봤다. 이 논문은 탄소도 함께 고려하도록 시스템을 바꾸자는 것이다.                                                                                                                                                                                                                                                                                                                                                                                         |
| 원문 | <i>In recent years, an extensive research has been conducted on carbon-aware resource management in cloud computing systems. These studies address dynamic workload management by exploiting temporal and/or spatial variability of carbon intensity of electricity of data centers, as well as taking advantage of the fact that many types of batch workloads, such as AI/ML training jobs, have a substantial temporal and geographical flexibility.</i> |
| 해석 | 최근 몇 년간, 클라우드 컴퓨팅 시스템의 탄소인지 자원관리에 대한 광범위한 연구가 수행됐다. 이 연구들은 데이터 센터 전기의 탄소집약도의 시간적·공간적 변동성을 활용하고, AI/ML 학습 잡 같은 많은 유형의 배치 워크로드가 상당한 시간적·지리적 유연성을 갖는다는 사실을 이용해 동적 워크로드 관리를 다룬다.                                                                                                                                                                                                                                                                                |
| 설명 | 배치 워크로드(batch workload)란? 실시간이 아니라 나중에 처리해도 되는 작업들. AI 모델 학습이 대표적이다. "지금 당장" 끝낼 필요가 없고 내일까지만 끝내면 되니까, 가장 저렴하거나 깨끗한 시간대로 미룰 수 있다. <b>이건 CAST와 유사</b><br>시간적 유연성: 지금 당장 안 해도 된다 → 더 깨끗한 전기가 나오는 시간으로 미룰 수 있다.<br>지리적 유연성: 어느 서버에서 해도 된다 → 더 깨끗한 전기를 쓰는 지역 서버로 보낼 수 있다.                                                                                                                                                                                        |
| 원문 | <i>For example, Google traces indicate that a high percentage of jobs submitted to the Borg scheduler have a low priority and fall in the free and best-effort-batch tiers with weak Service Level Agreements (SLAs) [18].</i>                                                                                                                                                                                                                              |
| 해석 | 예를 들어, 구글 트레이스 데이터를 보면 Borg 스케줄러에 제출된 작업 중 상당수가 낮은 우선순위를 가지며 SLA가 약한 free-best-effort-batch 티어에 속한다[18].                                                                                                                                                                                                                                                                                                                                                    |
| 설명 | <b>Borg</b> : 구글이 내부적으로 사용하는 거대한 작업 스케줄러. 전 세계 구글 서버를 관리한다.<br><b>SLA(Service Level Agreement)</b> : "이 작업은 몇 시간 안에 끝내야 한다"는 약속. SLA가 약하다 = 마감에 느슨하다 = 미뤄도 된다.<br>구글조차도 전체 작업 중 상당 부분은 급하지 않은 작업이다. 이걸 깨끗한 전기가 있을 때로 미루면 된다!                                                                                                                                                                                                                                |
| 원문 | <i>Similar characteristics are observed for jobs in Meta. For example, in Meta, about 87% of offline data processing workloads have completion time Service Level Objectives (SLOs) more than four hours, with the majority having 24-hour SLOs. This offers considerable flexibility for adjusting the schedule of workloads based on the availability of low carbon energy [1].</i>                                                                       |
| 해석 | 메타(Meta, 구 페이스북)에서도 유사한 특성이 관찰된다. 예를 들어 메타에서 오프라인 데이터 처리 워크로드의 약 87%가 완료 시간 SLO가 4시간 이상이며, 대다수가 24시간 SLO를 가진다. 이는 저탄소 에너지 가용성에 기반해 워크로드 스케줄을 조정하는 데 상당한 유연성을 제공한다[1].                                                                                                                                                                                                                                                                                       |
| 설명 | 메타(인스타그램·페이스북 운영사)도 전체 작업의 87%는 4시간 이상의 여유가 있다. 24시간 여유 있는 작업도 대부분이다.<br><b>결론</b> : 실제 세계에서 "미뤄도 되는 작업"이 압도적으로 많다. 이 여유를 이용해서 CO <sub>2</sub> 를 줄이자!                                                                                                                                                                                                                                                                                                       |
| 원문 | <i>Enabling temporal and spatial shifting of workloads requires the deployment of dynamic scheduling algorithms, which take real time data about power consumption, carbon intensity of electricity, and workload characteristics and make efficient scheduling and placement decisions.</i>                                                                                                                                                                |
| 해석 | 워크로드의 시간적·공간적 이동을 가능하게 하려면, 전력 소비, 전기의 탄소집약도, 워크로드 특성에 대한 실시간 데이터를 받아 효율적인 스케줄링·배치 결정을 내리는 동적 스케줄링 알고리즘을 배포해야 한다.                                                                                                                                                                                                                                                                                                                                           |
| 설명 | "언제, 어디서 할지"를 실시간으로 결정하려면 알고리즘이 필요하다. 매 시간마다 "지금 어느 나라 전기가 제일 깨끗한가? 이 작업의 마감은 언제인가?"를 계산해서 최선의 선택을 해야 한다.                                                                                                                                                                                                                                                                                                                                                   |
| 원문 | <i>A body of research has considered temporal shifting of workloads employing optimization techniques such as linear programming, matching, and greedy techniques [2], [4], [8], [10], [15]. These studies considered various performance metrics in their objective functions such as electricity price, carbon price, carbon-intensity of grid energy, and QoS.</i>                                                                                       |
| 해석 | 시간 이동(temporal shifting)을 다루는 상당수의 연구가 선형계획법, 매칭, 탐욕 기법 등의 최적화 기법을 활용했다 [2],[4],[8],[10],[15]. 이 연구들은 전기 요금, 탄소 가격, 그리드 에너지의 탄소집약도, QoS 등 다양한 성능 지표를 목적함수에 고려했다.                                                                                                                                                                                                                                                                                            |
| 설명 | 기존 연구들은 "언제 할지(시간 이동)"만 다뤘다. "어디서 할지(공간 배치)"는 별도로 연구됐다.                                                                                                                                                                                                                                                                                                                                                                                                     |
| 원문 | <i>In addition to temporal shifting, many studies considered the spatial workload placement among geographically distributed data centers [3], [16], [19], as well as distributed web services [17]. These efforts optimize various objective functions including minimizing carbon footprint, and/or maximizing the profit of cloud provider while guaranteeing QoS of workloads. They</i>                                                                 |

employed optimization techniques such as linear programming, distributed algorithms, reinforcement learning, and federated learning.

**해석** 시간 이동 외에도, 많은 연구들이 지리적으로 분산된 데이터센터들[3],[16],[19] 및 분산 웹 서비스[17] 간의 공간적 워크로드 배치를 고려했다. 이 연구들은 탄소발자국 최소화, 클라우드 사업자의 수익 최대화, QoS 보장 등 다양한 목적함수를 최적화한다. 선형계획법, 분산 알고리즘, 강화학습, 연합학습 등의 최적화 기법을 활용했다.

**설명** 공간 배치 연구: "어느 나라 서버에서 할지"만 다뤘다.

**이 논문의 차별점:** 시간 이동 + 공간 배치를 동시에! 게다가 실제 쿠버네티스 시스템에 구현까지 했다.

**원문** Combining temporal and spatial workload shifting, we introduce Caspian1, a carbon-aware workload scheduler in multi-cluster Kubernetes2 environments. In summary, our main contributions are as follows.

**해석** 시간적 이동과 공간적 이동을 결합하여, 우리는 멀티클러스터 쿠버네티스 환경을 위한 탄소인지 워크로드 스케줄러인 Caspian을 소개한다. 요약하면, 우리의 주요 기여는 다음과 같다.

**설명** 각주1: Caspian 작업은 T. Bahreini가 IBM T.J. Watson Research Center에 있을 때 수행됐다.

각주2: <https://kubernetes.io>

- **[기여 1]** 우리는 다목적 최적화 문제를 도입한다: (1) 저탄소 에너지가 있을 때·있는 곳에서 워크로드를 실행하여 탄소 배출을 최소화; (2) 워크로드의 평균 지연(latency)을 최소화; (3) 워크로드의 평균 완료 시간을 최소화.  
→ 탄소, 지연, 완료시간 세 가지 목표를 동시에 잡는 수학적 문제를 정의했다.
- **[기여 2]** 우리는 최적화 문제의 복잡도를 해결하는 효율적인 알고리즘을 설계하고 Caspian에 배포한다.  
→ 이 문제가 너무 어렵기 때문에(NP-hard), 빠르게 충분히 좋은 답을 찾는 알고리즘을 만들었다.
- **[기여 3]** 우리는 Caspian이 멀티클러스터 관리 플랫폼과 협력하여 대상 클러스터에 결정을 적용하는 프레임워크를 설계한다.  
→ 실제로 쓸 수 있는 시스템 구조를 만들었다.
- **[기여 4]** 우리는 Caspian을 구현하고 실제 쿠버네티스 멀티클러스터 환경에 배포한다.  
→ 논문에서 이론으로 끝내지 않고 실제로 작동하는 소프트웨어를 만들었다.
- **[기여 5]** 우리는 실세계 탄소집약도 데이터와 다양한 특성의 워크로드를 사용하여, Caspian이 QoS만 고려하는 베이스라인 스케줄러에 비해 높은 탄소 절감을 달성함을 보인다.  
→ 실험으로 효과를 증명했다.

-----

## II. System Model and Problem Statement (시스템 모델 및 문제 정의)

|    |                                                                                                                                                                                                                                                                                                                                                                         |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>Caspian is a batch scheduler in a multi-cluster Kubernetes environment which decides when and where workloads are executed, based on the sustainability aspects of the clusters such as carbon intensity of energy sources and server power efficiency. Figure 1 shows a high level architecture of Caspian and its interactions in a multi-cluster environment.</i> |
| 해석 | <b>Caspian</b> 은 에너지원의 탄소집약도 및 서버 전력 효율과 같은 클러스터의 지속가능성 측면에 기반하여, 워크로드가 언제, 어디서 실행될지를 결정하는 멀티클러스터 쿠버네티스 환경의 배치 스케줄러다. Figure 1은 <b>Caspian</b> 의 고수준 아키텍처와 멀티클러스터 환경에서의 상호작용을 보여준다.                                                                                                                                                                                     |
| 설명 | 배치 스케줄러: "이 작업을 지금 바로 실행"이 아니라 "언제, 어디서 실행할지 계획"을 세우는 역할. 음식 배달로 치면 어느 라이더에게 어느 시간에 배달을 맡길지 배정하는 사람과 같다.                                                                                                                                                                                                                                                                |

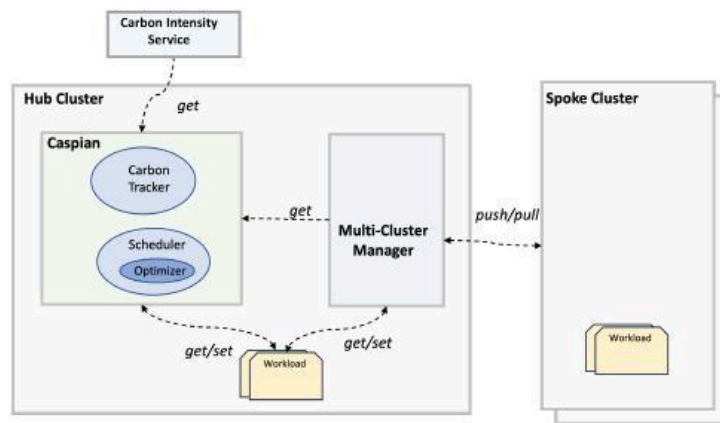


Fig. 1: A high level architecture of Caspian in a multi-cluster environment.

Fig. 1: Caspian의 고수준 아키텍처 — 허브 클러스터 안의 Caspian이 멀티클러스터 관리자를 통해 스포크 클러스터들을 제어한다.

|    |                                                                                                                                                                                                                                                                                                                                                                                                                       |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>We consider a hub-spoke configuration consisting of a master (hub) cluster, as a multi-cluster control plane, and several worker (spoke) clusters, as target clusters managed by the hub to run workloads. Caspian, as a scheduling and placement controller, lives in the hub, interacting with a multi-cluster manager which handles queuing, dispatching, and execution of workloads in the spoke clusters.</i> |
| 해석 | 우리는 멀티클러스터 컨트롤 플레인 역할을 하는 마스터(허브) 클러스터와, 허브가 관리하며 워크로드를 실행하는 여러 워커(스포크) 클러스터로 구성된 허브-스포크 구성을 고려한다. 스케줄링·배치 컨트롤러인 <b>Caspian</b> 은 허브에 위치하며, 스포크 클러스터에서 워크로드의 큐잉, 디스패치, 실행을 처리하는 멀티클러스터 매니저와 상호작용한다.                                                                                                                                                                                                                   |
| 설명 | <b>허브-스포크 구조</b> : 허브(중앙)는 "관제탑", 스포크(바퀴살)는 실제 GPU가 있는 "현장 서버"들이다. 자전거 바퀴처럼 중앙(허브)에서 여러 방향으로 뻗어 있다.<br><b>스포크는 MCAD라는 실서비스?</b><br>이 논문의 실험에서는 허브 1개 + 스포크 3개(캐나다, 일본, 독일)다.<br><b>Caspian</b> 은 허브에만 있고, 직접 스포크에 접근하지 않는다. 항상 멀티클러스터 매니저(MCAD)를 통해서만 명령을 전달한다.                                                                                                                                                       |

|    |                                                                                                                                                                                                                                                                                                                                                   |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>Caspian consists of two main components: a carbon tracker and a scheduler. The carbon tracker (1) gets the geographical location of the spoke clusters from the multi-cluster manager and (2) fetches periodically the predicted values of carbon intensity for those locations from external sources, such as a carbon intensity service.</i> |
| 해석 | <b>Caspian</b> 은 탄소 추적기(carbon tracker)와 스케줄러, 두 가지 주요 구성요소로 이루어진다. 탄소 추적기는 (1) 멀티클러스터 매니저로부터 스포크 클러스터의 지리적 위치를 얻고, (2) 외부 소스(예: 탄소집약도 서비스)로부터 해당 위치의 예측 탄소집약도 값을 주기적으로 가져온다.                                                                                                                                                                     |
| 설명 | <b>Carbon Tracker(탄소 추적기)</b> : "지금 각 나라의 전기가 얼마나 깨끗한가?"를 실시간으로 감시하는 역할. Electricity Maps(app.electricitymaps.com)라는 외부 서비스에서 데이터를 가져온다.<br>Caspian 자체가 탄소 데이터를 만드는 게 아니라, 외부 API에서 데이터를 받아서 사용한다.                                                                                                                                                |

|    |                                                                                                                                                                                                                                                                                                                                                                                 |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>The scheduler runs periodically. At the beginning of each period, it takes the information of spoke clusters (such as carbon intensity, power efficiency, and resource availability), as well as characteristics of workloads (such as run time, resource requirement, and deadline), and determines the best scheduling and placement for workloads using an optimizer.</i> |
| 해석 | 스케줄러는 주기적으로 실행된다. 각 주기 시작 시 스포크 클러스터 정보(탄소집약도, 전력 효율, 자원 가용성 등)와 워크로드 특성(실행 시간, 자원 요구량, 마감기한 등)을 받아, 옵티마이저를 사용해 워크로드의 최적 스케줄링·배치를 결정한다.                                                                                                                                                                                                                                         |
| 설명 | <p>스케줄러가 하는 일(매 주기마다):</p> <ol style="list-style-type: none"> <li>① 각 서버(스포크)의 현재 상태 확인: "지금 얼마나 바쁜가? 전기는 깨끗한가?"</li> <li>② 대기 중인 작업 목록 확인: "어떤 작업이 있고, 각각 마감이 언제인가?"</li> <li>③ 최적 계획 수립: "어떤 작업을 어느 서버에서 언제 시작할까?"</li> </ol>                                                                                                                                                 |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>Once these decisions are made, Caspian updates the scheduling specifications of workloads' Kubernetes manifest files, notifying the multi-cluster manager about the scheduling decisions for the next time slot. Next, the multi-cluster manager applies these decisions in the destination clusters and updates the status of workloads accordingly. More details about the multi-cluster manager and its interactions with Caspian is provided in Section IV.</i> |
| 해석 | 결정이 내려지면 Caspian은 워크로드의 쿠버네티스 매니페스트 파일의 스케줄링 사양을 갱신하여, 다음 시간 슬롯에 대한 스케줄링 결정을 멀티클러스터 매니저에 알린다. 그러면 멀티클러스터 매니저는 목적지 클러스터에 이 결정을 적용하고 그에 따라 워크로드 상태를 갱신한다. 멀티클러스터 매니저와 Caspian의 상호작용에 대한 더 자세한 내용은 Section IV에 제공된다.                                                                                                                                                                                                                                                      |
| 설명 | <p>쿠버네티스 매니페스트: 작업에 대한 설명서 파일(YAML 형식). "이 작업은 GPU 4개가 필요하고, 독일 서버에서 실행해줘" 같은 내용을 담는다.</p> <p>Caspian은 이 파일에서 딱 두 가지 필드만 수정한다: 어디서 실행할지(targetCluster), 지금 실행할지 말지(dispatchingGates).</p>                                                                                                                                                                                                                                                                              |

## A. Problem Statement (문제 정의)

|    |                                                                                                                                                                                                                                                                                                                                                                                                |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>This section describes the optimization problem that is addressed by the optimizer in Caspian. The multi-cluster environment considered here consists of a hub cluster and <math>M</math> spoke clusters. We consider a discrete time slotted system in which the decisions are made periodically, based on the status of the system, over a time horizon of <math>T</math> time slots.</i> |
| 해석 | 이 섹션은 Caspian의 옵티마이저가 다루는 최적화 문제를 설명한다. 여기서 고려하는 멀티클러스터 환경은 허브 클러스터 1개와 M개의 스포크 클러스터로 구성된다. 우리는 시스템 상태에 기반하여 T개의 시간 슬롯에 걸쳐 주기적으로 결정이 내려지는 이산 시간 슬롯 시스템을 고려한다.                                                                                                                                                                                                                                  |
| 설명 | <p>이산 시간 슬롯: 시간을 연속적으로 보지 않고 1시간 단위로 끊어서 본다. 예를 들어 1시간=슬롯1, 2시간=슬롯2, ... 이런 식으로 관리한다.</p> <p><b>이거는 동규방식</b></p> <p><math>M</math> = 스포크 클러스터 수 (실험에서는 3개)    <b>아마도 실제환경이 제한적이지 않을까</b></p> <p><math>T</math> = 전체 시간 지평선 (몇 시간 앞까지 계획할지)</p>                                                                                                                                                 |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>We denote by <math>N</math> the set of workloads. Each workload <math>i</math> is represented by the tuple <math>(a_{ij}, r_i, l_i, d_i)</math>. Attribute <math>a_{ij} \in \{0, 1\}</math> is the policy of workload <math>i</math> for cluster <math>j</math>; it is zero if workload <math>i</math> is not allowed to be executed on cluster <math>j</math>; and one otherwise. Attribute <math>r_i</math> is the required amount of compute resources, <math>l_i</math> is the run time of execution of workload <math>i</math>, and <math>d_i</math> is the deadline.</i> |
| 해석 | 워크로드 집합을 $N$ 으로 표기한다. 각 워크로드 $i$ 는 튜플 $(a_{ij}, r_i, l_i, d_i)$ 로 표현된다. 속성 $a_{ij} \in \{0,1\}$ 은 워크로드 $i$ 의 클러스터 $j$ 에 대한 정책으로, 워크로드 $i$ 가 클러스터 $j$ 에서 실행 불가이면 0, 그렇지 않으면 1이다. 속성 $r_i$ 는 필요한 컴퓨팅 자원량, $l_i$ 는 워크로드 $i$ 의 실행 시간, $d_i$ 는 마감기한이다.                                                                                                                                                                                                                                                                                                                                 |
| 설명 | <p>각 작업(워크로드)을 4개의 숫자로 표현한다:</p> <ul style="list-style-type: none"> <li>• <math>a_{ij}</math>: 작업 <math>i</math>를 서버 <math>j</math>에서 실행할 수 있는가? (0=불가, 1=가능). 예: 특정 데이터가 일본에만 있어서 일본 서버에서만 가능한 경우 <math>\rightarrow a_{\text{일본}}=1</math>, 나머지는 0.</li> <li>• <math>r_i</math>: 필요한 GPU 개수. 예: AI 학습에 GPU 4개 필요.</li> <li>• <math>l_i</math>: 실행 시간. 예: 이 작업은 3시간 걸림.</li> <li>• <math>d_i</math>: 마감기한. 예: 내일 오전 9시까지 끝내야 함.</li> </ul> <p><b>데이터도 고려했군</b></p>                                                                                                                |

|    |                                                                                                                                                                                                                                                                                                                                                                            |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>In this formulation, we consider a single type of compute resource, but the formulation could be easily extended to multiple types. Here, meeting the deadline of a workload is a soft constraint, i.e., the optimizer is not forced to schedule the workload before its deadline; but violating the deadline has an impact on the value of the objective function.</i> |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|    |                                                                                                                                                                                                                                                              |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 해석 | 이 정식화에서 우리는 단일 유형의 컴퓨팅 자원만 고려하지만, 정식화는 여러 유형으로 쉽게 확장할 수 있다. 여기서 워크로드의 마감기한 준수는 연성 제약(soft constraint)이다. 즉 옵티마이저가 마감기한 이전에 반드시 스케줄해야 하는 것은 아니지만, 마감기한을 어기면 목적함수 값에 영향을 미친다.                                                                                  |
| 설명 | <p>단일 자원 유형: 이 논문에서는 GPU만 고려. 실제로는 CPU, 메모리 등도 있지만 단순화.</p> <p>연성 제약(soft constraint): "꼭 지켜야 한다"가 아니라 "어기면 점수가 깎인다"는 방식.</p> <p>왜 연성 제약을 쓰나? — 만약 "절대 마감을 넘지면 안 된다"(경성 제약)고 하면, 자원이 부족할 때 아예 해가 존재하지 않는다. 연성 제약은 늦더라도 일단 스케줄하고 점수가 깎이는 방식이라 항상 해가 존재한다.</p> |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | We denote the set of spoke clusters by $M$ . Each cluster $j$ is represented by the tuple $(C_j^t, I_j^t, p_j)$ , where $C_j^t$ is the available computing resources and $I_j^t$ is the carbon intensity during time slot $t$ in cluster $j$ . Attribute $p_j$ is an estimate of power efficiency in cluster $j$ , where $C_j^t$ is the available computing resources and $I_j^t$ is the carbon intensity during time slot $t$ in cluster $j$ . Attribute $p_j$ is an estimate of power efficiency in cluster $j$ . We assume that power consumption is linear in resource utilization for all clusters [6]. The dynamic power consumption is estimated as the product of resource utilization and the constant factor $p_j$ . Thus, the carbon emission associated with the dynamic power consumption in cluster $j$ in time slot $t$ is the product of the resource utilization, power consumption factor $p_j$ , and carbon intensity $I_j^t$ . |
| 해석 | 스포크 클러스터 집합을 $M$ 으로 표기한다. 각 클러스터 $j$ 는 튜플 $(C_j^t, I_j^t, p_j)$ 로 표현된다. $C_j^t$ 는 시간 슬롯 $t$ 에서 가용 컴퓨팅 자원이고, $I_j^t$ 는 시간 슬롯 $t$ 에서 클러스터 $j$ 의 탄소집약도다. 속성 $p_j$ 는 클러스터 $j$ 의 전력 효율 추정값이다. 우리는 모든 클러스터에서 전력 소비가 자원 사용률에 선형이라고 가정한다[6]. 동적 전력 소비는 자원 사용률과 상수 인자 $p_j$ 의 곱으로 추정된다. 따라서 시간 슬롯 $t$ 에서 클러스터 $j$ 의 동적 전력 소비에 따른 탄소 배출은 자원 사용률, 전력 소비 인자 $p_j$ , 탄소집약도 $I_j^t$ 의 곱이다.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 설명 | <p>각 서버(클러스터 <math>j</math>)도 3가지 숫자로 표현한다:</p> <ul style="list-style-type: none"> <li>• <math>C_j^t</math>: 시간 슬롯 <math>t</math>에서 가용 GPU 수. 시간마다 변할 수 있음.</li> <li>• <math>I_j^t</math>: 시간 슬롯 <math>t</math>에서의 탄소집약도 (<math>\text{gCO}_2/\text{kWh}</math>). 시간마다, 지역마다 다름.</li> <li>• <math>p_j</math>: 전력 효율 상수. 낮을수록 전력 효율이 좋음. 실험에서 일본(Spoke2)은 <math>p=100</math>, 나머지는 <math>p=200</math>.</li> </ul> <p>탄소 배출량 공식: 탄소 배출 = 자원 사용률 <math>\times p_j \times I_j^t</math>. 즉, 더 많이 쓰고, 효율이 나쁘고, 전기가 더러울수록 탄소가 많이 나온다.</p> <p>선형 전력 모델 가정[6]: GPU를 50% 사용하면 전력도 50% 소비한다는 단순화 가정. 실제로는 더 복잡하지만 근사로 사용.</p>                                                                                                                                                                                                                                                                                                                                  |

이것도 가정을 했네,  
만약 우리가 한다면 진짜로 가능하려나

|    |                                                                                                       |
|----|-------------------------------------------------------------------------------------------------------|
| 원문 | Further, we assume that job runtime is invariant to the cluster, as it may request specific hardware. |
| 해석 | 또한 우리는 작업이 특정 하드웨어를 요청할 수 있으므로, 작업의 실행 시간이 클러스터에 관계없이 일정하다고 가정한다.                                     |
| 설명 | 어느 서버에서 실행하든 같은 시간이 걸린다고 가정. 실제로는 서버마다 GPU 성능이 다를 수 있지만 단순화.                                          |

오에에엥

|    |                                                                                                                                                                                                                                                                                                                         |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | We address the problem of Multi-Objective Scheduling and Placement (MOSP) of workloads over multiple geographically distributed clusters. The decision variable is matrix $x$ of binary variables $x_{ij}^t$ , where $x_{ij}^t$ is one if workload $i$ starts at time slot $t$ in cluster $j$ ; and zero otherwise.     |
| 해석 | 우리는 지리적으로 분산된 여러 클러스터에 걸친 워크로드의 다목적 스케줄링 및 배치(MOSP) 문제를 다룬다. 결정 변수는 이진 변수 $x_{ij}^t$ 의 행렬 $x$ 이다. $x_{ij}^t = 1$ 은 워크로드 $i$ 가 시간 슬롯 $t$ 에 클러스터 $j$ 에서 시작한다는 의미이고, 0이면 그렇지 않다.                                                                                                                                           |
| 설명 | <p>MOSP(Multi-Objective Scheduling and Placement): 여러 목적을 동시에 최적화하는 스케줄링+배치 문제.</p> <p>결정 변수 <math>x_{ij}^t</math>: Caspian이 내려야 하는 결정을 수학적으로 표현한 것.</p> <p>예: <math>x_{\text{작업5\_일본\_3시간}} = 1 \rightarrow</math> "작업5를 일본 서버에서 3번째 시간 슬롯(3시간 후)에 시작해라"</p> <p>모든 작업·서버·시간 조합에 대해 0 또는 1을 결정하는 것이 Caspian의 핵심 과제.</p> |

|    |                                                                                                                                                                                                                                                                                                                                                                               |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | The primary goal of MOSP is to minimize the CFP due to the execution of workloads. To avoid infeasibility due to the limited resources, we do not force the optimizer to schedule all workloads; rather we encourage the optimizer to maximize resource utilization when and where low carbon energy is available. Thus, our first objective is to maximize $Z1(x)$ given by, |
| 해석 | MOSP의 주요 목표는 워크로드 실행으로 인한 CFP를 최소화하는 것이다. 제한된 자원으로 인한 실행 불가 상황을 피하기 위해, 우리는 옵티마이저가 모든 워크로드를 스케줄하도록 강제하지 않는다. 대신 저탄소 에너지가 있을 때·있는 곳에서 자원 사용률을 최대화하도록 권장한다. 따라서 첫 번째 목적은 $Z1(x)$ 를 최대화하는 것이다.                                                                                                                                                                                   |
| 설명 | 탄소를 최소화 = 깨끗한 전기가 있을 때 최대한 많이 일하자.                                                                                                                                                                                                                                                                                                                                            |

"모든 작업을 반드시 스케줄"이 아닌 이유: 자원이 부족하면 불가능한 해가 생겨 문제가 안 풀릴 수 있다. 그래서 "최대한 많이, 깨끗하게"로 바꿨다.

## ▶ 목적함수 $Z_1$ — 탄소 최소화 (식 1)

$$Z_1(\mathbf{x}) = \sum_{i \in \mathcal{M}} \sum_{t \in \mathcal{T}} \frac{\sum_{j \in \mathcal{N}} \sum_{t' \in \Delta_i^t} x_{ij}^{t'} \cdot r_i}{C_j^t \cdot I_j^t \cdot \rho_j} \quad (1)$$

작업  $i$ 가  $j$ 에서  $t$ 시간에 하라  
cast와 동일

식(1):  $Z_1(\mathbf{x})$  — 저탄소·고효율 구간에서 자원 사용을 최소화

**원문** In this formulation, quantity  $(\sum \sum x_{ij}^{t'} \cdot r_i) / C_j^t$  represents the resource utilization at time slot  $t$  in cluster  $j$ . The resource utilization of a cluster in a particular time slot is calculated as the ratio of resource requirement of all workloads running during the time slot over cluster capacity. Workload  $i$  is running on cluster  $j$  during time slot  $t$  if it is started in any time slots in the interval  $\Delta_i^t = \{t - l_i + 1, \dots, t\}$ .

**해석** 이 정식화에서,  $(\sum \sum x_{ij}^{t'} \cdot r_i) / C_j^t$ 는 시간 슬롯  $t$ 에서 클러스터  $j$ 의 자원 사용률을 나타낸다. 클러스터의 특정 시간 슬롯 자원 사용률은 해당 슬롯에 실행 중인 모든 워크로드의 자원 요구량 합을 클러스터 용량으로 나눈 비율이다. 워크로드  $i$ 가  $\Delta_i^t = \{t - l_i + 1, \dots, t\}$  구간의 임의 슬롯에서 시작했다면 시간 슬롯  $t$ 에 클러스터  $j$ 에서 실행 중인 것이다.

전체 GPU수를 알아야 하군

**설명** 자원 사용률: 지금 서버에서 쓰고 있는 GPU 수 ÷ 전체 GPU 수. 예: 32개 중 16개 사용 중이면 50%.

$\Delta_i^t = \{t - l_i + 1, \dots, t\}$ : 작업  $i$ 가 시간  $t$ 에 실행 중이라면, 최소  $l_i$  시간 슬롯 전에 시작했어야 한다. 예: 3시간짜리 작업이  $t=5$ 에 실행 중이라면  $t=3, 4, 5$  중 하나에 시작했어야 함.

$Z_1$ 의 핵심: 탄소집약도  $I$ 가 낮은(깨끗한) 시간-장소에서 자원을 많이 쓸수록  $Z_1$ 이 커진다. 이걸 최대화하면 자연스럽게 탄소가 줄어든다.

**원문** A second objective is concerned with the QoS of workloads. Here, the goal is to schedule workloads before their deadlines. However, due to the limited compute resources, some workloads may not meet their deadlines. To handle such cases, and also to avoid infeasible solutions, we assume that meeting deadlines is a soft constraint. Thus, instead of guaranteeing that all workloads are scheduled before their deadlines, we attempt to schedule as many workloads as possible while minimizing the sum of the lateness of workloads, or equivalently maximizing  $Z_2(\mathbf{x})$  given by,

**해석** 두 번째 목적은 워크로드의 QoS에 관한 것이다. 목표는 워크로드를 마감기한 이전에 스케줄하는 것이다. 그러나 제한된 자원으로 인해 일부 워크로드는 마감기한을 지키지 못할 수 있다. 이러한 경우와 실행 불가 해를 피하기 위해, 마감기한 준수를 연성 제약으로 가정한다. 따라서 모든 워크로드를 마감기한 전에 보장하는 대신, 워크로드의 지연 합을 최소화하면서( $Z_2(\mathbf{x})$  최대화) 가능한 한 많은 워크로드를 스케줄하려 한다.

**설명**  $Z_1$ 만 있으면? 탄소만 줄이고 마감은 무시하게 된다. 그러면 실용적이지 않다.  $Z_2$ 는 "제시간에 끝내기"를 같이 고려하게 만든다.

## ▶ 목적함수 $Z_2$ — 지연 최소화 (식 2), $y(i, t) = \max(1, l_i + t - d_i)$

attempt to schedule as many workloads as possible while minimizing the sum of the lateness of workloads, or equivalently maximizing  $Z_2(\mathbf{x})$  given by,

식(2):  $Z_2(\mathbf{x})$  — 마감기한 초과에 따른 페널티 최소화

**원문** where  $y(i, t)$  is a penalty function for lateness. This penalty function discourages the decision maker to schedule a workload after its deadline. A simple definition of the penalty function could be  $y(i, t) = \max(1, l_i + t - d_i)$ , which monotonically decreases the objective value with the increase of deviation of completion time of workload from its deadline.

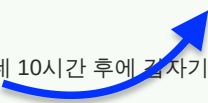
**해석**  $y(i, t)$ 는 지연에 대한 페널티 함수다. 이 페널티 함수는 의사결정자가 워크로드를 마감기한 이후에 스케줄하지 않도록 억제한다. 페널티 함수의 간단한 정의는  $y(i, t) = \max(1, l_i + t - d_i)$ 로, 완료 시각이 마감기한에서 벗어날수록 목적함수 값이 단조 감소한다.

**설명**  $y$  함수 이해하기:

- $t$ 에서 시작하면 완료 시각 =  $t + l_i$  (시작 시각 + 실행 시간)
- 마감 =  $d_i$
- $y = \max(1, \text{완료시각} - \text{마감}) = \max(1, l_i + t - d_i)$
- 마감 전에 끝나면:  $y = 1 \rightarrow x/y = x/1 = x$  (페널티 없음)
- 마감 2시간 후에 끝나면:  $y = 2 \rightarrow x/y = x/2$  (점수 반토막)
- 마감 10시간 후에 끝나면:  $y = 10 \rightarrow x/y = x/10$  (점수 10분의 1)

이건 소프트웨어조건  
그래서 시간 안에 했다는 거 언급 가능함

→ 늦으면 늦을수록 점수가 줄어드니까, 옵티마이저는 최대한 일찍 끝내려 한다.

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>A third objective controls the completion time of workloads. Scheduling the current workloads while not considering upcoming workloads in the future may result in inefficient utilization of compute resources. Let us consider a scenario in which the current workloads have extended deadlines. Under objective functions Z1 and Z2, the optimizer may decide to postpone the execution of workloads and schedule them later when low carbon energy is available, while a heavy demand may also arrive to the system in the future. This may cause some future workloads to miss their deadlines.</i> |
| 해석 | 세 번째 목적은 워크로드의 <u>완료 시간을 제어한다</u> . 미래의 워크로드를 고려하지 않고 현재 워크로드만 스케줄하면 컴퓨팅 자원이 비효율적으로 사용될 수 있다. 현재 워크로드가 마감기한이 여유로운 시나리오를 생각해보자. Z1과 Z2만으로는 옵티마이저가 저탄소 에너지가 있을 때까지 워크로드 실행을 계속 미룰 수 있는데, 그동안 미래에 무거운 수요가 시스템에 도착할 수도 있다. 이는 일부 미래 워크로드가 마감기한을 놓치게 할 수 있다.                                                                                                                                                                                                                                                                                                                                                      |
| 설명 | 왜 Z <sub>3</sub> 가 필요한가? — 구체적 예시:<br>지금 작업 A가 있고 마감이 24시간 후다. Z <sub>1</sub> , Z <sub>2</sub> 만 보면 옵티마이저는 "탄소가 제일 낮은 시간대까지 20시간 기다리자"고 할 수 있다.<br> <b>이 목적은 용량제한 두는것임 -&gt; 동규한테 좋을듯!</b><br>그런데 10시간 후에 갑자기 긴급 작업들이 쏟아지면? 서버가 꽉 차서 신규 긴급 작업들이 마감을 못 지킨다.<br>→ Z <sub>3</sub> 는 "지금 여유 있더라도 너무 미루지 마라"고 억제한다. 미래를 예측하지 않아도 자동으로 여유 자원을 남겨두는 효과.                                                                                                                                                                            |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>One approach to handle such cases is to schedule the current workloads while reserving resources for the predicted future load [3]. This approach requires the prediction of future load. In this paper, we assume that either the prediction of future load is not available or the future load is hard to predict. Consequently, we define a third objective function, Z3(x), which controls the completion time of workloads and does not let workloads be executed very late, even if they still have time ahead of their deadlines,</i> |
| 해석 | 이러한 경우를 처리하는 한 가지 접근법은 예측된 미래 부하를 위해 자원을 예약하면서 현재 워크로드를 스케줄하는 것이다[3]. 이 접근법은 미래 부하 예측이 필요하다. 이 논문에서는 미래 부하 예측이 불가능하거나 예측하기 어렵다고 가정한다. 따라서 우리는 워크로드의 완료 시간을 제어하고, 마감기한까지 시간이 남아있더라도 워크로드가 너무 늦게 실행되지 않도록 하는 세 번째 목적함수 Z3(x)를 정의한다.                                                                                                                                                                                                                                                                                                               |
| 설명 | 미래를 예측하는 방법도 있지만, 어렵다. 그래서 이 논문은 예측 없이 Z <sub>3</sub> 라는 수식 하나로 "자연스럽게 미루지 않는" 행동을 유도한다.                                                                                                                                                                                                                                                                                                                                                                                                                                                        |

▶ 목적함수 Z<sub>3</sub> — 완료시간 최소화 (식 3),  $\lambda(i, t) = t + l_i$

### 동규한테 좋을 조건

executed very late, even if they still have time ahead of their deadlines.

식(3): Z<sub>3</sub>(x) — 마감여유 있더라도 지나치게 늦지 않게

|    |                                                                                                                                                                                                                                                                                                                                                                                                                    |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>where <math>\lambda(i, t)</math> is the penalty function of completion time. Given this penalty function, the decision maker is encouraged to schedule workloads as soon as possible. Here, we simply consider the completion time as the penalty function, i.e., <math>\lambda(i, t) = t + l_i</math>. This function may also be defined by considering an estimation of arrival rate of future workloads.</i> |
| 해석 | $\lambda(i, t)$ 는 완료시간에 대한 페널티 함수다. 이 페널티 함수는 의사결정자가 가능한 한 빨리 워크로드를 스케줄하도록 독려한다. 여기서 우리는 완료시간 자체를 페널티 함수로 단순하게 정의한다. 즉 $\lambda(i, t) = t + l_i$ . 이 함수는 미래 워크로드 도착률의 추정값을 고려하여 정의할 수도 있다.                                                                                                                                                                                                                         |
| 설명 | $\lambda$ 함수: $\lambda = t + l_i$ (시작 시간 + 실행 시간 = 완료 시간) <b>여기도 그냥 시간에대한 점수를 줬군</b><br>늦게 시작할수록 $\lambda$ 가 커지고 $x/\lambda$ 가 작아진다 → 점수가 줄어든다.<br>→ 옵티마이저는 자연스럽게 일찍 시작하려 한다.<br>비유: 숙제를 일찍 낼수록 점수를 더 준다. 늦게 낼수록 점수가 깎인다. (마감은 안 넘겨도)                                                                                                                                                                              |

|    |                                                                                                                            |
|----|----------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>Given objective functions Z1(x), Z2(x), and Z3(x), we formulate the MOSP problem as an integer programming problem.</i> |
| 해석 | Z1(x), Z2(x), Z3(x) 목적함수를 바탕으로, 우리는 MOSP 문제를 정수계획법 문제로 정식화한다.                                                              |
| 설명 | 세 목적함수를 수식으로 조합해서 "수학적 최적화 문제"를 만든다.                                                                                       |

▶ MOSP 정식화 및 제약조건 (식 4~8)

considering an estimation of arrival rate of future workloads.

Given objective functions  $Z_1(\mathbf{x})$ ,  $Z_2(\mathbf{x})$ , and  $Z_3(\mathbf{x})$ , we formulate the MOSP problem as an integer programming problem.

$$\max_{\mathbf{x} \in X} (Z_1(\mathbf{x}), Z_2(\mathbf{x}), Z_3(\mathbf{x})) \quad (4)$$

subject to:

$$\sum_{i \in \mathcal{N}} \sum_{t' \in \Delta_i^t} x_{ij}^{t'} \cdot r_i \leq C_j^t \quad \forall j \in \mathcal{M}, t \in \mathcal{T}, \quad (5)$$

$$\sum_{j \in \mathcal{M}} \sum_{t \in \mathcal{T}} x_{ij}^t \leq 1 \quad \forall i \in \mathcal{N}, \quad (6)$$

$$x_{ij}^t \leq a_{ij} \quad \forall i \in \mathcal{N}, \forall j \in \mathcal{M}, t \in \mathcal{T}, \quad (7)$$

$$x_{ij}^t \in \{0, 1\} \quad \forall i \in \mathcal{N}, \forall j \in \mathcal{M}, t \in \mathcal{T}, \quad (8)$$

where  $X$  is the feasible set of decision matrices. Constraint (5)

식(4)~(8): MOSP 정식화 — 목적함수 + 제약조건 전체

|    |                                                                                                                                                                                                                                                                                                                                                                                                                               |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | where $X$ is the feasible set of decision matrices. Constraint (5) guarantees that for each cluster $j$ , the total amount of resources allocation cannot exceed its available capacity. Constraints (6)-(7) consider the policy of the workload for clusters and also ensure that each workload is scheduled on a maximum of one cluster. Constraint (8) represents the integrality requirements for the decision variables. |
| 해석 | $X$ 는 실행 가능한 결정 행렬의 집합이다. 제약 (5)는 각 클러스터 $j$ 에서 총 자원 배정이 가용 용량을 넘지 않음을 보장한다. 제약 (6)~(7)은 워크로드의 클러스터 정책을 고려하며, 각 워크로드가 최대 하나의 클러스터에만 스케줄됨을 보장한다. 제약 (8)은 결정 변수의 정수성 요건을 나타낸다.                                                                                                                                                                                                                                                  |
| 설명 | <p>제약 (5) — 자원 한도: 서버에 GPU가 32개 있으면, 동시에 32개 초과는 불가능하다.</p> <p>제약 (6) — 한 번만 실행: 같은 작업을 동시에 두 서버에서 실행할 수 없다. (<math>\leq 1</math>이라서 아예 실행 안 될 수도 있다 — 자원 부족 시)</p> <p>제약 (7) — 정책 준수: <math>a_{ij}=0</math>이면 그 클러스터에 배치 불가.</p> <p>제약 (8) — 정수 제약: <math>x_{ij}^t</math>는 0 또는 1만 가능. 0.5개 작업 같은 건 없다.</p>                                                                                                                  |

## B. Problem Complexity (문제의 복잡도)

|    |                                                                                                                                                                                                                                                                                    |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | The special case of MOSP with a single-objective function $\max Z_1(\mathbf{x})$ , $\mathbf{x} \in X$ , i.e., maximizing resource utilization when and where low-carbon energy is available, is NP-hard [3]. This implies that MOSP, with multiple objective functions is NP-hard. |
| 해석 | 단일 목적함수 $\max Z_1(\mathbf{x})$ , $\mathbf{x} \in X$ , 즉 저탄소 에너지가 있을 때·있는 곳에서 자원 사용률을 최대화하는 MOSP의 특수한 경우는 NP-hard이다[3]. 이는 다목적함수를 갖는 MOSP 역시 NP-hard임을 의미한다.                                                                                                                        |
| 설명 | <p>NP-hard란? "완벽한 최적해를 컴퓨터가 현실적인 시간 안에 구할 수 없는 문제 유형."</p> <p>예: 작업이 100개, 서버가 3개, 시간 슬롯이 40개라면 가능한 배치 경우의 수는 천문학적이다. 모든 경우를 다 따져보다가는 우주가 끝난다.</p> <p>→ 그래서 이 논문은 "완벽한 답" 대신 "충분히 좋은 답을 빠르게" 찾는 방법을 쓴다.</p>                                                                        |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | Furthermore, solving problems with multiple objective functions is not as straight forward as it is with single-objective problems. In multi-objective optimization problems with conflicting objectives, the goal is to find Pareto optimal solutions; the solution that could not be dominated by another feasible solution. It might be possible to obtain the Pareto optimal solutions of the problem when there is a few number of workloads in the system. But with a large number of workloads it is practically impossible to solve the problem in a reasonable amount of time. In the next section, we introduce an efficient algorithm to solve MOSP in polynomial time. |
| 해석 | 더 나아가, 다목적함수를 갖는 문제를 푸는 것은 단일 목적 문제만큼 단순하지 않다. 목적이 상충하는 다목적 최적화 문제에서의 목표는 파레토 최적해를 찾는 것이다. 파레토 최적해는 다른 실행 가능한 해에 의해 지배될 수 없는 해다. 시스템에 워크로드가 몇 개 없을 때는 파레토 최적해를 구하는 것이 가능할 수도 있다. 그러나 워크로드 수가 많으면 현실적인 시간 내에 문제를 푸는 것은 사실상 불가능하다. 다음 섹션에서 MOSP를 다항시간에 푸는 효율적인 알고리즘을 소개한다.                                                                                                                                                                                                                                                                                                                                                                                                           |
| 설명 | <p>목적이 상충한다는 게 무슨 뜻?</p> <p>탄소를 최소화하려면 → 작업을 미뤄야 한다.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |

마감을 지키려면 → 빨리 실행해야 한다.

이 두 목표가 서로 반대 방향이라 둘 다 완벽히 달성할 수 없다. 최적의 타협점을 찾는 게 목표다.

**파레토 최적:** "탄소를 더 줄이면 마감을 더 어기게 되고, 마감을 더 지키면 탄소가 더 나온다" 상태. 더 이상 한쪽을 희생하지 않고 다른 쪽을 개선할 수 없는 지점.

**다항시간:** 작업 수가 늘어도 계산 시간이 감당 가능하게 늘어나는 것. (NP-hard의 반대)

-----

### III. Caspian Algorithms (알고리즘)

|    |                                                                                                                                                                                                                                                                                                                                                                                                                       |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>The scheduler determines the schedule time as well as the target cluster for execution of workloads. The scheduler works in a timely manner where the length of the interval between its invocations is determined based on the characteristics of workloads such as QoS requirement and the average execution time of workloads. A high level description of the scheduler algorithm is given in Algorithm 1.</i> |
| 해석 | 스케줄러는 워크로드의 스케줄 시간과 실행 대상 클러스터를 결정한다. 스케줄러는 시간에 맞게 작동하며, 호출 사이의 간격 길이는 QoS 요구사항, 워크로드의 평균 실행 시간 같은 워크로드 특성에 따라 결정된다. 스케줄러 알고리즘의 고수준 설명이 Algorithm 1에 주어진다.                                                                                                                                                                                                                                                            |
| 설명 | 스케줄러는 계속 반복 실행된다. $\delta$ (델타) 간격마다 "지금 상황을 보고 최선의 계획을 세운다."<br>간격 $\delta$ 는 상황에 따라 다르게 설정할 수 있다. 마감이 빠빠하면 짧게, 여유 있으면 길게.                                                                                                                                                                                                                                                                                           |

#### ▶ Algorithm 1 — Scheduler( $\delta$ ): 전체 스케줄러의 반복 루프

##### Algorithm 1 Scheduler ( $\delta$ )

```
1: while true do
2:   ( $\mathcal{M}, \mathcal{N}$ )  $\leftarrow$  getClusterAndWorkloadInfo()
3:    $y \leftarrow$  optimizer( $\mathcal{M}, \mathcal{N}$ )
4:   for each  $i \in \mathcal{N}$  do
5:     if isValid( $y[i]$ ) then
6:       setTarget( $i, y[i]$ )
7:     else
8:       suspend( $i$ )
```

Algorithm 1: Scheduler —  $\delta$  간격마다 실행되는 컨트롤러 루프

#### Algorithm 1 줄별 상세 해설

| 줄 | 코드                                                                      | 초등학생도 OK 설명                                                                                                                                                                                               |
|---|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | while true do                                                           | 무한 반복 시작. Caspian은 끄지 않는 이상 영원히 돌아간다. 마치 신호등 컨트롤러처럼 계속 상황을 보고 결정을 내린다.                                                                                                                                    |
| 2 | ( $\mathcal{M}, \mathcal{N}$ ) $\leftarrow$ getClusterAndWorkloadInfo() | 현재 상황 조회. $\mathcal{M}$ = 모든 스포크 클러스터 정보(현재 탄소집약도, 남은 GPU 수 등), $\mathcal{N}$ = 현재 대기 중이거나 실행 중인 모든 작업 목록. 마치 "지금 어느 창고가 비어있고, 처리할 택배가 뭐가 있지?"를 확인하는 것.                                                   |
| 3 | $y \leftarrow$ optimizer( $\mathcal{M}, \mathcal{N}$ )                  | 핵심: 최적 배치 계산. 옵티마이저(Algorithm 2)를 호출해서 "각 작업을 어느 서버에서 언제 시작할지" 계획을 짠다. 결과 $y$ 는 각 작업에 대한 지정 서버 정보다. $y[i] = j$ 이면 "작업 $i$ 는 클러스터 $j$ 에서 지금 시작해라"는 뜻. $y[i] = -1$ 이면 "이 작업은 이번 슬롯은 건너뛰어라(다음 기회를 기다려라)"는 뜻. |
| 4 | for each $i \in \mathcal{N}$ do                                         | 모든 작업에 대해 하나씩 처리. 대기 중인 모든 작업 $i$ 를 순서대로 본다.                                                                                                                                                              |
| 5 | if isValid( $y[i]$ ) then                                               | 배치 결정이 유효한가? $y[i]$ 가 실제 클러스터 번호라면( $\neq -1$ 이 아니라면) "이 작업은 이번 슬롯에 실행"이라는 뜻이다.                                                                                                                           |
| 6 | setTarget( $i, y[i]$ )                                                  | 목적지 서버를 지정. 작업 $i$ 의 쿠버네티스 매니페스트 파일에서 targetCluster 필드를 $y[i]$ 로 설정한다. 그리고 dispatchingGates에서 sustainability gate를 제거해서 "이제 실행해도 돼"라고 MCAD에 알린다.                                                          |
| 7 | else                                                                    | 배치 결정이 -1인 경우 (이번 슬롯엔 실행 안 함).                                                                                                                                                                            |
| 8 | suspend( $i$ )                                                          | 작업을 일시 보류. dispatchingGates에 "sustainability gate"를 추가해서 MCAD에 "이 작업 아직 실행하지 마"라고 알린다. 작업은 큐에 남아서 다음 슬롯을 기다린다.                                                                                            |
| 9 | wait( $\delta$ )                                                        | $\delta$ 시간만큼 기다린 후 처음으로 돌아간다. 이 기간 동안 탄소집약도가 바뀌고, 새 작업이 들어오고, 실행 중인 작업이 끝날 수 있다. 그러면 다시 줄 1부터 반복.                                                                                                        |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>The input to the scheduler is <math>\delta</math>, the length of the interval between periods. At the beginning of each period, the scheduler gets all available spoke clusters as well as the workloads (Line 2) and saves their characteristics in sets <math>\mathcal{M}</math> and <math>\mathcal{N}</math>, respectively. Then, it calls the optimizer to find the best scheduling and placement for the workloads (Line 3).</i> |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|    |                                                                                                                                                                             |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 해석 | 스케줄러의 입력은 $\delta$ , 즉 주기 사이의 간격 길이다. 각 주기 시작 시 스케줄러는 사용 가능한 모든 스포크 클러스터와 워크로드를 가져와(2번 줄) 특성을 각각 집합 M과 N에 저장한다. 그런 다음 옵티마이저를 호출해 워크로드의 최적 스케줄링·배치를 찾는다(3번 줄).               |
| 설명 | $\delta$ 는 예를 들어 1시간. 즉 1시간마다 한 번씩 전체 계획을 다시 세운다.<br>원문: "The input to the scheduler is $\delta$ , the length of the interval between periods." — $\delta$ 는 주기 사이의 간격 길이다. |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>The scheduler has two choices. It could ask the optimizer to decide on the scheduling and placement of (1) the queued workloads or (2) the running workloads, by revisiting its previously made decisions, in addition to the queued workloads. In this implementation, the scheduler considers the second option and asks optimizer to decide about all workloads in the system, those currently running in the spoke clusters as well as workloads queued in the hub cluster.</i> |
| 해석 | 스케줄러는 두 가지 선택지가 있다. (1) 대기 중인 워크로드에 대해서만 결정하거나, (2) 이전에 내린 결정을 재검토 하며 실행 중인 워크로드에 대해서도 결정하는 옵션이다. 이 구현에서는 두 번째 옵션을 사용하여, 스포크 클러스터에서 현재 실행 중인 워크로드와 허브 클러스터에서 대기 중인 워크로드 모두에 대해 옵티마이저가 결정하도록 한다.                                                                                                                                                                                                                                                                                        |
| 설명 | 선택 2 (이미 실행 중인 것도 재고려): 만약 실행을 시작했더니 갑자기 캐나다 전기가 더러워지고 독일이 깨끗해졌다면? 실행 중인 작업을 중단하고 독일로 옮기는 결정도 할 수 있다.<br>→ 변화에 유연하게 대응할 수 있지만, 계산량이 더 많다.                                                                                                                                                                                                                                                                                                                                              |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>The output of the optimizer is a vector <math>y</math> which indicates the destination cluster for each workload. Here, <math>y_i = j</math> means that workload <math>i</math> is scheduled on cluster with index <math>j</math> and <math>y_i = -1</math> means that workload <math>i</math> is scheduled for a time period later than the current time period. For workloads with a valid target cluster index, the scheduler sets the target cluster field in the scheduling specification of the workload Kubernetes manifest, notifying the multi-cluster manager where to dispatch them. For workloads that are not scheduled for the upcoming time slot, the scheduler sets the sustainability gate in the scheduling specification of the workload Kubernetes manifest, notifying the multi-cluster manager to suspend them (Lines 4-8).</i> |
| 해석 | 옵티마이저의 출력은 각 워크로드의 목적지 클러스터를 나타내는 벡터 $y$ 다. $y_i = j$ 는 워크로드 $i$ 가 인덱스 $j$ 인 클러스터에 스케줄됐다는 의미이고, $y_i = -1$ 은 워크로드 $i$ 가 현재 시간 슬롯보다 나중에 스케줄됐다는 의미다. 유효한 목적지 클러스터 인덱스가 있는 워크로드의 경우, 스케줄러는 워크로드의 쿠버네티스 매니페스트 스케줄링 사양에서 목적지 클러스터 필드를 설정하여 멀티클러스터 매니저에 디스패치 위치를 알린다. 다음 시간 슬롯에 스케줄되지 않은 워크로드의 경우, 스케줄러는 쿠버네티스 매니페스트 스케줄링 사양에 sustainability gate를 설정하여 멀티클러스터 매니저에 보류를 알린다(4-8번 줄).                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 설명 | $y$ 벡터: 각 작업에 대한 "어디로 보낼지" 답안지. 예: $y = [\text{일본}, -1, \text{캐나다}, \text{독일}, -1, \dots]$<br>$y[i] = -1$ : "아직 때가 아니야. 기다려." → suspend(보류)<br>$y[i] = 2$ : "작업 $i$ 는 Spoke2(일본)로 보내." → setTarget<br>원문: " $y_i = -1$ means that workload $i$ is scheduled for a time period later than the current time period." — $y_i = -1$ 이면 이번 슬롯이 아닌 다음 슬롯으로 예정됐다는 뜻.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>The optimizer solves the MOSP optimization problem with three objectives: <math>Z_1(x)</math>, <math>Z_2(x)</math>, and <math>Z_3(x)</math>. Many techniques for solving multi-objective optimization problems are based on converting the original problem into a single objective problem [5], known as scalarization. Consequently, we construct a single objective function by summing the individual objective functions, each multiplied by a coefficient, as <math>Z(x) = \omega_1 \cdot Z_1(x) + \omega_2 \cdot Z_2(x) + \omega_3 \cdot Z_3(x)</math>, where <math>\omega_k</math> is the normalized value of <math>\omega_k</math>, the preference value for objective function <math>Z_k(x)</math>.</i> |
| 해석 | 옵티마이저는 세 가지 목적: $Z_1(x)$ , $Z_2(x)$ , $Z_3(x)$ 로 MOSP 최적화 문제를 푼다. 다목적 최적화 문제를 푸는 많은 기법은 원 문제를 단일 목적함수 문제로 변환하는 스칼라화[5]에 기반한다. 따라서 우리는 각 목적함수에 계수를 곱해 더한 단일 목적함수 $Z(x) = \omega_1 \cdot Z_1(x) + \omega_2 \cdot Z_2(x) + \omega_3 \cdot Z_3(x)$ 를 구성한다. 여기서 $\omega_k$ 는 목적함수 $Z_k(x)$ 의 선호 값 $\omega_k$ 의 정규화된 값이다.                                                                                                                                                                                                                                                                                                                                                                                                  |
| 설명 | 스칼라화(Scalarization): 세 목표를 하나의 점수로 합치는 방법.<br>원문 수식: $Z(x) = \omega_1 \cdot Z_1(x) + \omega_2 \cdot Z_2(x) + \omega_3 \cdot Z_3(x)$<br>여기서 $\omega_k$ 는 $\omega_k$ 의 정규화된 값(normalized value of $\omega_k$ ), 즉 선호도 값이다.<br>예: "탄소"에 50점, "마감"에 30점, "완료시간"에 20점 가중치를 주면 총점 = $0.5 \times Z_1 + 0.3 \times Z_2 + 0.2 \times Z_3$ .<br>$\omega_1, \omega_2, \omega_3$ : 운영자가 설정하는 가중치. $\omega_1$ 이 크면 탄소를 더 중요시, $\omega_1$ 이 작으면 마감을 더 중요시.<br>$\omega_k$ : $\omega_k$ 를 정규화한 것 (단위를 맞추기 위해).                                                                                                                                                                                                                       |

최적화 및 가중치 문제로 바꿨군

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | Due to the inconsistent units of the objective functions, normalization of the preferences of objectives is necessary. Normalization plays a significant role in achieving the optimal solutions with the given preferences for objectives. Several studies have focused on various normalization methods [9]. However, the most efficient approaches consider normalizing by the magnitude of the optimal values, where each optimal value is obtained by solving an optimization problem with a single objective function. Since it is not possible to find the optimal solution for each single objective in a reasonable amount of time, we consider the LP relaxation of each problem and obtain coefficient $\theta_k = 1/\bar{Z}_k$ , where $\bar{Z}_k$ is the optimal objective value obtained by solving the LP relaxation of the problem with the single objective $Z_k$ . |
| 해석 | 목적함수들의 단위가 서로 달라 정규화가 필요하다. 정규화는 주어진 선호도로 최적해를 달성하는 데 중요한 역할을 한다. 여러 연구들이 다양한 정규화 방법에 집중했다[9]. 그러나 가장 효율적인 접근법은 각 단일 목적함수로 최적화 문제를 풀어 얻은 최적값의 크기로 정규화하는 것이다. 각 단일 목적의 최적해를 현실적인 시간 내에 구하는 것이 불가능하므로, 각 문제의 LP 완화를 고려하여 계수 $\theta_k = 1/\bar{Z}_k$ 를 구한다. 여기서 $\bar{Z}_k$ 는 단일 목적 $Z_k$ 로 LP 완화 문제를 풀어 얻은 최적 목적값이다.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 설명 | <p>왜 정규화가 필요한가? <math>Z_1</math>는 수백만 단위, <math>Z_2</math>는 수십 단위일 수 있다. 그냥 더하면 <math>Z_1</math>이 결과를 독점한다. 정규화로 각각 "최대로 잘했을 때를 1"로 만들어 공평하게 비교한다.</p> <p><b>LP 완화(LP Relaxation):</b> 원래 <math>x</math>는 0 또는 1(정수)이어야 하는데, 0.3, 0.7 같은 소수도 허용하면 훨씬 빨리 풀 수 있다. 이 완화된 문제를 LP(선형계획법)로 풀어서 <math>\theta_k</math>를 구한다. <b>가볍다는 이야기 첨언 가능</b></p> <p>계수 <math>\theta_k = 1/\bar{Z}_k</math>: <math>\bar{Z}_k</math>는 단일 목적 <math>Z_k</math>로 LP 완화를 풀었을 때의 최적값. <math>\theta_k</math>는 그 역수로, 정규화 계수로 쓰인다.</p> <p>→ 진짜 최적값은 너무 오래 걸려서 못 구하니까, LP 완화의 답을 대신 쓴다.</p>                                                                                                                                                                                                                                                                                                                      |

|    |                                                                                                                                                                                 |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | Even with this simplification, the scheduling and placement problem is NP-hard and we need to find an efficient algorithm that solves the problem in reasonable amount of time. |
| 해석 | 이 단순화를 해도 스케줄링·배치 문제는 NP-hard이며, 현실적인 시간에 문제를 푸는 효율적인 알고리즘이 필요하다.                                                                                                               |
| 설명 | LP 완화를 썼어도 여전히 어렵다. 그래서 Algorithm 2가 LP + 탐욕 라운딩을 조합한다.                                                                                                                         |

#### ▶ Algorithm 2 — Optimizer(M, N): LP 완화 + 탐욕 라운딩

**Algorithm 2 Optimizer( $\mathcal{M}, \mathcal{N}$ )**

```

1:  $\_, \theta_k \leftarrow \text{LPSolver}(Z_k, \mathcal{M}, \mathcal{N}) \quad \forall k \in \{1, 2, 3\}$ 
2:  $\bar{\omega}_k \leftarrow \frac{\omega_k}{\theta_k} \quad \forall k \in \{1, 2, 3\}$ 
3:  $Z(\mathbf{x}) \leftarrow \sum_{k \in \{1, 2, 3\}} \bar{\omega}_k \cdot Z_k(\mathbf{x})$ 
4:  $\bar{\mathbf{x}}, _ \leftarrow \text{LPSolver}(Z, \mathcal{M}, \mathcal{N})$ 
5:  $x_{ij}^t \leftarrow 0 \quad \forall i \in \mathcal{N}, j \in \mathcal{M}, t \in \mathcal{T}$ 
6:  $y_i \leftarrow -1 \quad \forall i \in \mathcal{N}$ 
7:  $b_i \leftarrow \sum_{j \in \mathcal{M}} \sum_{t \in \mathcal{T}} \bar{x}_{ij}^t \quad \forall i \in \mathcal{N}$ 
8: Sort workloads in non-increasing order of  $b_i$ .
9: for each  $i \in \mathcal{N}$  do
10:    $(j^*, t^*) \leftarrow \text{Allocation}(i, \mathbf{x}, Z, \mathcal{M})$ 
11:   if isValid( $j^*$ ) then
12:      $x_{ij^*}^{t^*} \leftarrow 1$ 
13:     if  $t^* == 1$  then
14:        $y_i \leftarrow j^*$ 
15: Output:  $\mathbf{y}$ 

```

relaxation of each problem and obtain coefficient  $\theta_k = 1/\bar{Z}_k$ , where  $\bar{Z}_k$  is the optimal objective value obtained by solving the LP relaxation of the problem with the single objective  $Z_k$ . Even with this simplification, the scheduling and placement

Algorithm 2: Optimizer — LP 분수해 → 정수해 변환

#### Algorithm 2 줄별 상세 해설

| 줄 | 코드                                                                                                      | 초등학생도 OK 설명                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|---|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | $\_, \theta_k \leftarrow \text{LPSolver}(Z_k, \mathcal{M}, \mathcal{N})$<br>$\forall k \in \{1, 2, 3\}$ | <p><b>[정규화 계수 계산]</b> <math>Z_1, Z_2, Z_3</math> 각각을 혼자 최대화하면 얼마까지 가능한가? LP 완화를 사용해 목적함수 <math>Z_1</math>만으로 최대화하면 <math>Z_1</math> 값이 나온다. 마찬가지로 <math>Z_2</math>만, <math>Z_3</math>만 풀어서 <math>\bar{Z}_2, \bar{Z}_3</math>를 구한다.</p> <p><math>\theta_k = 1/\bar{Z}_k \rightarrow</math> 정규화 계수. 이 값으로 나중에 세 목표의 스케일을 맞춘다.</p> <p>왜? <math>Z_1</math>의 최대값이 1000이고 <math>Z_2</math>의 최대값이 50이면, 가중합할 때 <math>Z_1</math>이 20배 더 영향을 준다. 불공평하다. <math>\theta_k</math>로 나누면 둘 다 "최대 1점" 기준으로 통일된다.</p> |

| 줄  | 코드                                                                       | 초등학생도 OK 설명                                                                                                                                                                                                            |
|----|--------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2  | $\omega_k \leftarrow \omega_k / \theta_k$<br>$\forall k \in \{1, 2, 3\}$ | <b>[정규화된 가중치 계산]</b><br>운영자가 설정한 가중치 $\omega_k$ 에 정규화 계수 $\theta_k$ 를 적용한다.<br>$\omega_k = \omega_k \times Z_k$ ( $\theta_k = 1/Z_k$ 이므로)<br>→ 이제 $\omega_k$ 가 각 목표의 실제 중요도를 반영하면서 스케일도 맞춰진 최종 가중치다.                   |
| 3  | $Z(x) \leftarrow \sum \omega_k \cdot Z_k(x)$                             | <b>[통합 목적함수 구성]</b><br>세 목적함수를 정규화된 가중치로 합쳐서 하나의 점수 $Z(x)$ 로 만든다.<br>$Z(x) = \omega_1 \cdot Z_1(x) + \omega_2 \cdot Z_2(x) + \omega_3 \cdot Z_3(x)$<br>이제부터는 이 단일 점수 $Z(x)$ 를 최대화하는 것이 목표다.                            |
| 4  | $\bar{x}, \leftarrow \text{LPSolver}(Z, M, N)$                           | <b>[LP 완화로 분수해 획득]</b><br>통합 목적함수 $Z(x)$ 를 LP 완화로 푼다. $x$ 는 0 또는 1이어야 하지만, 여기서는 0~1 사이 소수도 허용한다.<br>결과 $\bar{x}$ 에는 "작업 3을 일본에 0.7, 독일에 0.3 배치" 같은 말이 안 되는 값이 나온다.<br>하지만 이게 어디가 좋은지 힌트를 준다! "일본에 0.7"이면 일본이 더 좋다는 신호. |
| 5  | $x_{ij}^{*t} \leftarrow 0$<br>$\forall i \in N, j \in M, t \in T$        | <b>[정수 해 행렬 초기화]</b><br>실제로 쓸 정수 해 행렬 $x$ 를 모두 0으로 초기화. 이제 여기에 1을 하나씩 채워 넣을 것이다.<br>"아직 아무 작업도 아무 서버에도 배정 안 됨" 상태.                                                                                                     |
| 6  | $y_i \leftarrow -1 \ \forall i \in N$                                    | <b>[출력 벡터 초기화]</b><br>모든 작업에 대해 "이번 슬롯에는 실행 안 함(-1)"으로 초기화. 이후 배치가 확정되면 해당 값이 클러스터 번호로 바뀐다.                                                                                                                            |
| 7  | $b_i \leftarrow \sum \sum \bar{x}_{ij}^{*t}$<br>$\forall i \in N$        | <b>[작업별 LP배정 총합 계산]</b><br>각 작업 $i$ 에 대해, LP 완화 해 $\bar{x}$ 에서 모든 서버·모든 시간에 걸쳐 배정된 값의 합을 구한다.<br>$b_i$ = 작업 $i$ 가 LP 해에서 얼마나 강하게 "배정되고 싶어하는가"의 지표.<br>예: $b_i = 0.9$ 이면 LP가 이 작업을 강하게 배정하고 싶어함. $b_i = 0.1$ 이면 약하게.    |
| 8  | Sort workloads by $b_i$<br>(non-increasing)                              | <b>[우선순위 정렬]</b><br>$b_i$ 가 높은 작업(LP가 강하게 원하는 작업)부터 먼저 배치한다.<br>왜? LP가 "이 작업이 중요하다"고 판단한 순서를 존중해서 자원을 먼저 준다.                                                                                                           |
| 9  | for each $i \in N$ do                                                    | <b>우선순위 순서대로 작업 배치 시작.</b><br>$b_i$ 내림차순으로 정렬된 작업들을 하나씩 처리한다.                                                                                                                                                          |
| 10 | $(j^*, t^*) \leftarrow \text{Allocation}$<br>$(i, x, Z, M)$              | <b>[실제 자리 찾기]</b><br>작업 $i$ 를 위한 최적의 (서버, 시간슬롯) 조합을 찾는다. Algorithm 3가 이 역할을 한다.<br>$j^*$ = 찾은 서버, $t^*$ = 찾은 시작 시간. 못 찾으면 $j^* = -1$ .                                                                                 |
| 11 | if isValid( $j^*$ ) then                                                 | <b>유효한 자리를 찾았는가?</b> $j^* \neq -1$ 이면 자리가 있다는 뜻.                                                                                                                                                                       |
| 12 | $x_{ij}^{*t^*} \leftarrow 1$                                             | <b>[배치 확정]</b><br>"작업 $i$ 를 서버 $j^*$ 에서 시간 $t^*$ 에 시작"으로 확정. 이후 다른 작업들은 이 자원이 쓰인다는 걸 알고 계획을 짤다.                                                                                                                        |
| 13 | if $t^* == 1$ then                                                       | <b>이번 슬롯(<math>t^*=1</math>)에 시작해야 하는가?</b><br>$t^*=1$ 이면 "지금 바로 시작"이라는 뜻. $t^*$ 가 더 크면 미래 슬롯에 예약만 해두는 것.                                                                                                              |
| 14 | $y_i \leftarrow j^*$                                                     | <b>[출력에 기록]</b><br>이번 슬롯에 시작하는 작업만 $y$ 에 서버 번호를 기록한다. 나중에 Algorithm 1이 이 $y$ 를 받아서 setTarget을 호출한다.                                                                                                                    |
| 15 | Output: $y$                                                              | <b>[결과 반환]</b><br>"각 작업을 이번 슬롯에 어느 서버에서 시작할지 (-1이면 보류)" 목록을 Algorithm 1으로 돌려준다.                                                                                                                                        |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <p>Algorithm 2 describes the optimizer. First, the optimizer calls the LP-Solver to find the optimal LP solution with each of the objective functions. The output of LPSolver is the scheduling matrix and the objective value of the solution. Here, since we are only interested in the objective values, i.e., <math>\theta_1, \theta_2, \theta_3</math>, we do not save the scheduling matrix (Line 1). Then, the optimizer, based on the LP solution formulates the objective function <math>Z(x)</math> and calls the LPSolver; this time to obtain LP solution for the MOSP problem with <math>Z(x)</math> as the objective function (Lines 2-4). The solution is saved in matrix <math>\bar{x}</math>.</p> |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|    |                                                                                                                                                                                                                                                                                                                                  |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 해석 | Algorithm 2는 옵티마이저를 설명한다. 먼저 옵티마이저는 각 목적함수로 LP 최적 해를 찾기 위해 LP-Solver를 호출한다. LPSolver의 출력은 스케줄링 행렬과 해의 목적값이다. 여기서는 목적값 $\theta_1, \theta_2, \theta_3$ 만 필요하므로 스케줄링 행렬은 저장하지 않는다(1번 줄). 그런 다음 옵티마이저는 LP 해를 바탕으로 목적함수 $Z(x)$ 를 구성하고 LPSolver를 다시 호출한다. 이번에는 $Z(x)$ 를 목적함수로 MOSP 문제의 LP 해를 구하기 위함이다(2-4번 줄). 해는 행렬 $\bar{x}$ 에 저장된다. |
| 설명 | Algorithm 2의 전체 흐름을 한 줄로: "먼저 각 목표를 따로따로 풀어서 기준점을 잡고, 그 기준으로 세 목표를 합친 문제를 다시 풀어서 분수해를 얻는다."                                                                                                                                                                                                                                      |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | Next, it builds a feasible solution $x$ from the fractional solution $\bar{x}$ . For this purpose, we initialize an allocation matrix $x$ to zero (Line 5). We also initialize target vector $y$ to save the target cluster for those workloads that are scheduled for the upcoming time slot (Line 6). The algorithm sorts the workloads in non-increasing order of their total allocation value, $b_i$ , obtained by LPSolver (Lines 7-8). Then it chooses an unallocated workload with the maximum value of $b_i$ and calls allocation algorithm to find the best time slot and cluster to fit the workload. If a valid spot $(j^*, t^*)$ is found, the algorithm updates the allocation matrix and target vector and proceeds to the next candidate workload (Lines 9-14). |
| 해석 | 다음으로, 분수해 $\bar{x}$ 에서 실행 가능한 정수 해 $x$ 를 구성한다. 이를 위해 배치 행렬 $x$ 를 0으로 초기화한다(5번 줄). 다음 시간 슬롯에 스케줄될 워크로드의 목적지 클러스터를 저장하기 위한 목적지 벡터 $y$ 도 초기화한다(6번 줄). 알고리즘은 LPSolver가 구한 총 배정값 $b_i$ 의 내림차순으로 워크로드를 정렬한다(7-8번 줄). 그런 다음 $b_i$ 가 최대인 미배정 워크로드를 선택해 배치 알고리즘을 호출하여 워크로드를 넣을 최적의 시간 슬롯과 클러스터를 찾는다. 유효한 자리 $(j^*, t^*)$ 가 발견되면 배치 행렬과 목적지 벡터를 갱신하고 다음 후보 워크로드로 진행한다(9-14번 줄).                                                                                                                                                                                                                                                                                                                                                                                                          |
| 설명 | 전체 흐름 비유: 시험장 좌석 배치와 같다.<br>① LP가 "학생들이 이 자리에 대략 이 비율로"라는 힌트를 준다.<br>② LP에서 "이 자리에 제일 앉고 싶어하는" 학생( $b_i$ 가 큰)부터 순서대로 진짜 자리에 앉힌다.<br>③ 한 명씩 Algorithm 3을 불러서 "이 학생한테 딱 맞는 자리 어디야?" 찾는다.<br>④ 자리가 확정되면 다음 학생으로 넘어간다.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |

▶ Algorithm 3 — Allocation( $i, x, Z, M$ ): 개별 작업의 최적 자리 찾기

Algorithm 3 Allocation( $i, x, Z, M$ )

```

1: Sort all  $(j, t) \in M \times T$  in non-increasing order of  $z_{ij}^t$ .
2:  $(j^*, t^*) \leftarrow (-1, -1)$ 
3: for each  $(j, t) \in M \times T$  do
4:   if  $\text{fit}(i, x, j, t') \quad \forall t' \in \{t, \dots, t + l_i - 1\}$  then
5:      $(j^*, t^*) \leftarrow (j, t)$ 
6:   break

```

Algorithm 3: Allocation — 목적함수 기여도 높은 자리부터 탐욕 탐색

Algorithm 3 줄별 상세 해설

| 줄 | 코드                                                                     | 초등학생도 OK 설명                                                                                                                                                                                                                                                                          |
|---|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Sort all $(j, t) \in M \times T$<br>by $z_{ij}^t$ desc                 | <b>[후보 자리 우선순위 정렬]</b><br>작업 $i$ 를 놓을 수 있는 모든 (서버 $j$ , 시작시간 $t$ ) 조합을 점수 $z_{ij}^t$ 기준으로 내림차순 정렬한다.<br>$z_{ij}^t$ : "작업 $i$ 를 서버 $j$ 에서 시간 $t$ 에 시작할 때 목적함수 $Z$ 에 기여하는 점수"<br>→ 점수가 높다 = 탄소도 적고 마감도 잘 지키는 좋은 선택이다.<br>예: (캐나다, $t=2$ )=9점, (일본, $t=5$ )=7점, (독일, $t=3$ )=4점 순으로 정렬. |
| 2 | $(j^*, t^*) \leftarrow (-1, -1)$                                       | <b>[초기화]</b> 아직 자리를 못 찾은 상태로 시작. $j^*=-1, t^*=-1$ 은 "자리 없음"을 의미한다.                                                                                                                                                                                                                   |
| 3 | for each $(j, t) \in M \times T$ do                                    | 점수 높은 자리부터 순서대로 시도.<br>1번 줄에서 정렬한 순서대로, 점수가 제일 좋은 (서버, 시간) 조합부터 차례로 시도한다.                                                                                                                                                                                                            |
| 4 | if $\text{fit}(i, x, j, t')$<br>$\forall t' \in \{t, \dots, t+l_i-1\}$ | <b>[실행 가능성 검사]</b><br>작업 $i$ 를 서버 $j$ 에서 시간 $t$ 에 시작할 수 있는가?<br><b>핵심 조건 1 (제약 5):</b> 작업 $i$ 가 실행되는 동안(시간 $t$ 부터 $t+l_i-1$ 까지) <u>매 시간 슬롯마다</u> 서버 $j$ 에 GPU가 충분히 남아있어야 한다.<br>예: 3시간짜리 작업을 $t=2$ 에 시작하면 $t=2, t=3, t=4$ 에 모두 GPU가 있어야 한                                            |

| 줄 | 코드                             | 초등학생도 OK 설명                                                                                                                                              |
|---|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
|   |                                | 다. 하나라도 부족하면 실패.<br><b>핵심 조건 2 (제약 7):</b> $a_{ij} = 1$ 이어야 한다 (이 서버에서 실행 허용).                                                                           |
| 5 | $(j^*, t^*) \leftarrow (j, t)$ | <b>[자리 확정!]</b><br>조건을 만족하는 첫 번째 (서버, 시간) 조합을 찾았다. $j^*$ 와 $t^*$ 에 기록한다.                                                                                 |
| 6 | break                          | <b>즉시 중단.</b> 첫 번째로 가능한 자리를 찾으면 바로 멈춘다. 더 좋은 자리가 있어도 찾지 않는다.<br>이것이 <b>탐욕(Greedy) 알고리즘</b> 의 특징: "지금 당장 가장 좋은 것"을 선택하고 더 이상 고민하지 않는다. 최적이지 않을 수 있지만 빠르다. |
| 7 | Output: $(j^*, t^*)$           | <b>[결과 반환]</b><br>찾은 (서버, 시작시간)을 Algorithm 2로 반환한다. 못 찾으면 (-1, -1)을 반환해서 Algorithm 2가 이 작업을 보류 처리한다.                                                     |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <p>Algorithm 3 describes the allocation algorithm which finds the best target cluster and best schedule time for each workload. The input to this algorithm are the candidate workload <math>i</math>, the current allocation matrix <math>x</math>, the objective function <math>Z</math>, and the set of spoke clusters <math>M</math>. First, the algorithm sorts all pairs of (target cluster <math>j</math>, scheduled time slot <math>t</math>) for workload <math>i</math> in descending order of <math>z_{ij} \cdot t</math> values (Line 1). Next, the algorithm tries to find the first pair of <math>(j, t)</math> in which the workload could be fitted (Lines 3-7). Here, <math>fit(i, x, j, t)</math> checks if workload <math>i</math> with current allocation <math>x</math> is schedulable in time slot <math>t</math> and cluster <math>j</math> or not. For this purpose, the fit procedure checks if both constraints 5 and 7 are satisfied.</p> |
| 해석 | <p><b>Algorithm 3</b>은 각 워크로드의 최적 목적지 클러스터와 최적 스케줄 시간을 찾는 배치 알고리즘이다. 이 알고리즘의 입력은 후보 워크로드 <math>i</math>, 현재 배치 행렬 <math>x</math>, 목적함수 <math>Z</math>, 스포크 클러스터 집합 <math>M</math>이다. 먼저 알고리즘은 워크로드 <math>i</math>에 대해 모든 (목적지 클러스터 <math>j</math>, 스케줄 시간 슬롯 <math>t</math>) 쌍을 <math>z_{ij} \cdot t</math> 값의 내림차순으로 정렬한다(1번 줄). 다음으로 알고리즘은 워크로드를 넣을 수 있는 첫 번째 <math>(j, t)</math> 쌍을 찾으려 한다(3-7번 줄). 여기서 <math>fit(i, x, j, t)</math>는 현재 배치 <math>x</math>에서 워크로드 <math>i</math>가 시간 슬롯 <math>t</math>와 클러스터 <math>j</math>에서 스케줄 가능한지 확인한다. 이를 위해 <math>fit</math> 절차는 제약 5와 7이 모두 만족되는지 확인한다.</p>                                                                                                                                                                                                                                                                                                                                                               |
| 설명 | <p><b>Algorithm 3 전체 비유:</b> 맛집을 예약하는 것과 같다.</p> <ol style="list-style-type: none"> <li>① 내가 원하는 식당 목록을 "맛있는 순서"로 정렬한다 (<math>z_{ij} \cdot t</math> 내림차순 정렬)</li> <li>② 제일 원하는 식당에 전화: "4명에서 저녁 6시에 갈 수 있어요?" (<math>fit</math> 검사)</li> <li>③ "자리 있어요" 하면 바로 예약! (<math>j^*, t^*</math> 확정, break)</li> <li>④ "자리 없어요" 하면 두 번째 식당으로 넘어간다.</li> <li>⑤ 모든 식당이 안 되면 "오늘은 못 가겠다" (-1, -1 반환)</li> </ol> <p><b>탐욕 알고리즘 한계:</b> 지금 당장 가장 좋아 보이는 것을 선택하지만, 전체적으로 최적이지 않을 수 있다. 예: 점수 9점인 캐나다에 먼저 배치했더니, 나중에 점수 8점짜리 중요한 작업이 캐나다 자원을 못 써서 6점 독일로 가게 될 수 있다. 하지만 빠르다는 장점이 있다.</p>                                                                                                                                                                                                                                                                                                                                                                                            |

## IV. Implementation (구현)

|    |                                                                                                                                                                                                                                                                                                                                                        |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>Caspian4 is an open source Kubernetes controller written in Golang. Caspian uses Multi-Cluster App Dispatcher (MCAD)5 as a workload queuing and multi-cluster management platform to dispatch and handle workloads in the destination clusters. In this section, we describe MCAD in more detail and elaborate on its interaction with Caspian.</i> |
| 해석 | <b>Caspian</b> 은 <b>Golang</b> 으로 작성된 오픈소스 쿠버네티스 컨트롤러다(각주4). <b>Caspian</b> 은 목적지 클러스터에서 워크로드를 디스패치하고 처리하기 위한 워크로드 큐잉·멀티클러스터 관리 플랫폼으로 <b>MCAD</b> (각주5)를 사용한다. 이 섹션에서는 <b>MCAD</b> 를 더 자세히 설명하고 <b>Caspian</b> 과의 상호작용을 설명한다.                                                                                                                          |
| 설명 | 각주4: <a href="https://github.com/sustainablecomputing/caspian">https://github.com/sustainablecomputing/caspian</a> (실제 코드를 직접 볼 수 있다)<br>각주5: <a href="https://github.com/project-codeflare/mcad">https://github.com/project-codeflare/mcad</a><br>Golang: 구글이 만든 프로그래밍 언어. 빠르고 가볍다. <b>여기 실제 코드 볼 수 있는</b>                                            |

### A. Multi Cluster App Dispatcher (MCAD)

|    |                                                                                                                                                                                                                                                                                                                                                                                                          |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>MCAD is a Kubernetes controller that enables the deployment of batch jobs in multi-cluster environments. It provides mechanisms to guarantee that sufficient resources are available in the destination clusters before creating pods for executing workloads.</i>                                                                                                                                    |
| 해석 | <b>MCAD</b> 는 멀티클러스터 환경에서 배치 잡 배포를 가능하게 하는 쿠버네티스 컨트롤러다. <b>MCAD</b> 는 워크로드 실행을 위한 파드를 생성하기 전에 목적지 클러스터에 충분한 자원이 있음을 보장하는 메커니즘을 제공한다.                                                                                                                                                                                                                                                                     |
| 설명 | <b>파드(Pod)</b> : 쿠버네티스에서 실제로 프로그램이 실행되는 컨테이너 묶음. 작업 하나 = 파드 하나(이상).<br><b>MCAD</b> 는 자원이 부족하면 파드를 만들지 않는다. 자원이 준비됐을 때만 시작한다.                                                                                                                                                                                                                                                                             |
| 원문 | <i>MCAD deploys an AppWrapper (AW) custom resource to wrap any Kubernetes object including service, job, deployment, and other custom resources defined by the user. AW custom resource helps MCAD to support batch jobs and gang scheduling in Kubernetes clusters. Kubernetes objects within an AW stay in the queue until the total requested resources are available in the destination cluster.</i> |
| 해석 | <b>MCAD</b> 는 서비스, 잡, 디플로이먼트, 사용자 정의 리소스 등 모든 쿠버네티스 오브젝트를 감싸기 위해 <b>AppWrapper(AW)</b> 커스텀 리소스를 배포한다. <b>AW</b> 커스텀 리소스는 <b>MCAD</b> 가 쿠버네티스 클러스터에서 배치 잡과 갱 스케줄링을 지원하도록 돕는다. <b>AW</b> 내의 쿠버네티스 오브젝트는 목적지 클러스터에서 요청된 총 자원이 가용해질 때까지 큐에 머문다.                                                                                                                                                              |
| 설명 | <b>AppWrapper(AW)</b> : 작업을 포장하는 박스. 박스 안에 실제 실행할 것들이 들어있다.<br><b>갱 스케줄링(gang scheduling)</b> : AI 분산 학습처럼 여러 GPU가 동시에 필요한 작업. GPU 4개가 필요한데 3개만 있으면 시작 안 하고 4개가 다 준비될 때까지 기다린다.<br><b>AW</b> 내의 모든 것이 한꺼번에 준비되어야 실행이 시작된다.                                                                                                                                                                               |
| 원문 | <i>At the AW level, a user can specify resource requirement for each object, expected run time of the workload, the deadline for the completion time of the workload, and the policies for the clusters. In addition to workload characteristics, there are scheduling specification fields in AW to support scheduling and placement in multi-cluster environment.</i>                                  |
| 해석 | <b>AW</b> 수준에서 사용자는 각 오브젝트의 자원 요구사항, 워크로드의 예상 실행 시간, 완료 시간의 마감기한, 클러스터에 대한 정책을 지정할 수 있다. 워크로드 특성 외에도, 멀티클러스터 환경에서 스케줄링·배치를 지원하기 위한 스케줄링 사양 필드가 <b>AW</b> 에 있다.                                                                                                                                                                                                                                           |
| 설명 | <b>AW</b> 파일에는 "이 작업에 GPU 4개 필요, 3시간 걸림, 내일까지 완료" 같은 정보를 넣는다.                                                                                                                                                                                                                                                                                                                                            |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>One of these fields is dispatchingGates which indicates the list of gates that are placed by different placement engines on the workload. Each placement engine (such as Caspian) may have some policies on the workloads. For instance, Caspian may want to suspend an AW from execution due to sustainability concerns. To accomplish this, Caspian adds a sustainability gate to this field requesting MCAD to suspend the execution of a workload.</i> |
| 해석 | 이러한 필드 중 하나가 <b>dispatchingGates</b> 로, 서로 다른 배치 엔진들이 워크로드에 걸어놓은 게이트 목록을 나타낸다. 각 배치 엔진( <b>Caspian</b> 등)은 워크로드에 대한 정책을 가질 수 있다. 예를 들어 <b>Caspian</b> 은 지속가능성 이유로 <b>AW</b> 의 실행을 보류하고 싶을 수 있다. 이를 위해 <b>Caspian</b> 은 <b>MCAD</b> 에 워크로드 실행 보류를 요청하는 <b>sustainability gate</b> 를 이 필드에 추가한다.                                                                                                                                                                |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 설명 | <p><b>dispatchingGates:</b> "이 작업 아직 실행하지 마" 신호들의 목록. 여러 엔진이 각각 게이트를 걸 수 있다.</p> <p>Caspian이 "sustainability gate"를 추가 → MCAD가 "게이트가 있으니 아직 실행 안 함" → 게이트 제거 → MCAD가 "이제 실행 해도 돼"</p> <p>마치 공사장 신호수가 "STOP" 패들을 들면 차가 안 가고, "GO" 패들을 들면 가는 것과 같다.</p>                                                                                                                                                                                                                                                                                                                                       |
| 원문 | <p>Another scheduling related field in AW is targetCluster which is set by the placement engines. For example, if the optimizer decides to place a workload on a specific cluster Spoke1, the scheduler in Caspian sets the targetCluster field in AW to Spoke1, notifying MCAD that this workload is scheduled on Spoke1. Then, MCAD will do its part in managing the life cycle of the workload in Spoke1.</p>                                                                                                                                                                            |
| 해석 | <p>AW의 또 다른 스케줄링 관련 필드는 targetCluster로, 배치 엔진이 설정한다. 예를 들어 옵티마이저가 워크로드를 Spoke1에 배치하기로 결정하면, Caspian의 스케줄러는 AW의 targetCluster 필드를 Spoke1로 설정하여 MCAD에 이 워크로드가 Spoke1에 스케줄됐음을 알린다. 그러면 MCAD가 Spoke1에서 워크로드의 생명주기 관리를 담당한다.</p>                                                                                                                                                                                                                                                                                                                                                                 |
| 설명 | <p>Caspian이 MCAD와 소통하는 방법은 단 두 가지다:</p> <p>① targetCluster 필드: "어디로 보낼지" (예: "Spoke1로 보내")</p> <p>② dispatchingGates: "지금 보낼지 말지" (게이트 추가=보류, 게이트 제거=실행)</p> <p>이 두 필드만 건드려서 MCAD 내부 코드 수정 없이 탄소인지 스케줄링을 구현했다. 매우 영리한 설계!</p>                                                                                                                                                                                                                                                                                                                                                               |
| 원문 | <p>MCAD works in two different modes: dispatcher mode and runner mode. In the hub cluster, MCAD runs in dispatcher mode (MCAD Dispatcher), while in each spoke cluster, it runs in runner mode (MCAD Runner). MCAD Dispatcher and MCAD Runner coordinate with each other to manage queuing, error handling, and execution of AWs through the Syncer component in MCAD.</p>                                                                                                                                                                                                                  |
| 해석 | <p>MCAD는 두 가지 다른 모드로 작동한다: 디스패처 모드와 러너 모드. 허브 클러스터에서는 MCAD가 디스패처 모드 (MCAD Dispatcher)로 실행되고, 각 스포크 클러스터에서는 러너 모드(MCAD Runner)로 실행된다. MCAD Dispatcher와 MCAD Runner는 MCAD의 Syncer 컴포넌트를 통해 AW의 큐잉, 오류 처리, 실행을 관리하기 위해서로 조율한다.</p>                                                                                                                                                                                                                                                                                                                                                             |
| 설명 | <p><b>MCAD Dispatcher(허브):</b> 교통 관제탑. 작업을 받아서 어느 스포크로 보낼지 결정하고 조율.</p> <p><b>MCAD Runner(스포크):</b> 현장 실행팀. 실제로 GPU에서 작업을 돌리고 상태를 보고.</p> <p><b>Syncer:</b> 허브-스포크 사이에서 정보를 동기화하는 역할. 스포크의 현재 상태를 허브로 올려주고(upsync), 허브의 결정을 스포크로 내려준다(push).</p>                                                                                                                                                                                                                                                                                                                                            |
| 원문 | <p>In addition to the AW, MCAD introduces the ClusterInfo custom resource, which represents the characteristics of clusters, such as resource availability, power consumption characteristics, geolocation, and carbon intensity. Some of these fields such as resource availability, power consumption characteristics, and geolocation are updated by the ClusterInfo Controller (another component of MCAD) in spoke clusters, while some other characteristics such as carbon intensity could be set by other external controllers such as the carbon tracker component in Caspian.</p> |
| 해석 | <p>AW 외에도 MCAD는 자원 가용성, 전력 소비 특성, 지리적 위치, 탄소집약도 등 클러스터의 특성을 나타내는 ClusterInfo 커스텀 리소스를 도입한다. 자원 가용성, 전력 소비 특성, 지리적 위치 같은 일부 필드는 스포크 클러스터에서 ClusterInfo Controller(MCAD의 또 다른 컴포넌트)가 갱신하고, 탄소집약도 같은 다른 특성은 Caspian의 탄소 추적기 컴포넌트 같은 외부 컨트롤러가 설정할 수 있다.</p>                                                                                                                                                                                                                                                                                                                                     |
| 설명 | <p><b>ClusterInfo:</b> 각 서버(클러스터)에 대한 정보 카드. "이 서버는 GPU 20개 남았고, 현재 탄소집약도는 300 gCO<sub>2</sub>/kWh"처럼.</p> <p>Caspian의 Carbon Tracker가 Electricity Maps에서 탄소집약도를 받아와서 ClusterInfo의 탄소집약도 필드를 업데이트한다.</p>                                                                                                                                                                                                                                                                                                                                                                                    |

## B. Integration of Caspian with MCAD

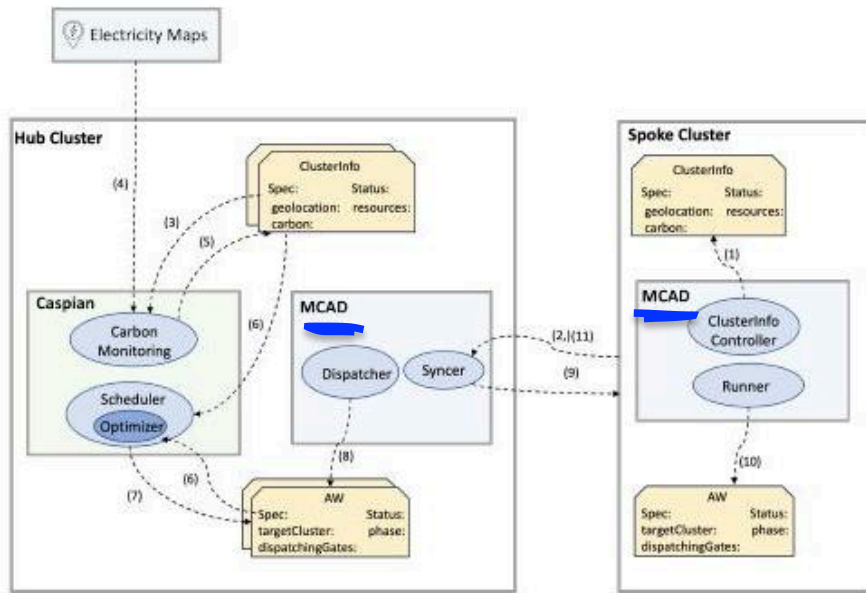


Fig. 2: Integration of Caspian with MCAD.

error handling, and execution of AWs through the Syncer component in MCAD. In addition to the AW, MCAD introduces the ClusterInfo custom resource, which represents the characteristics of clusters, such as resource availability, power consumption characteristics, geolocation, and carbon intensity. Some of these fields such as resource availability, power consumption characteristics, and geolocation are updated by the ClusterInfo Controller (another component of MCAD) in

Fig. 2: Caspian과 MCAD의 통합 — 번호순으로 11단계 상호작용이 이루어진다.

|    |                                                                                                                                                                                                                                                                                                                                                    |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | A summary of interactions between MCAD and Caspian is depicted in Figure 2. (1) In each spoke cluster, ClusterInfo controller periodically gets the geo-location of the cluster as well as the available resources in the cluster and updates ClusterInfo custom resource accordingly.                                                             |
| 해석 | MCAD와 Caspian 간의 상호작용 요약이 Figure 2에 묘사되어 있다. (1) 각 스포크 클러스터에서 ClusterInfo 컨트롤러는 클러스터의 지리적 위치 및 가용 자원을 주기적으로 가져와 ClusterInfo 커스텀 리소스를 갱신한다.                                                                                                                                                                                                         |
| 설명 | 스포크 서버가 스스로 "나 지금 GPU 20개 남았어, 위치는 일본이야"를 허브에 보고하는 것.                                                                                                                                                                                                                                                                                              |
| 원문 | (2) The Syncer in the hub cluster, periodically upsyns the objects from the spoke clusters to the hub cluster. This way, Caspian has access to the updated status of ClusterInfo as well as the status of AWs executing in the spoke clusters.                                                                                                     |
| 해석 | (2) 허브 클러스터의 Syncer는 스포크 클러스터의 오브젝트를 주기적으로 허브 클러스터로 업싱크한다. 이를 통해 Caspian은 ClusterInfo의 갱신된 상태와 스포크 클러스터에서 실행 중인 AW의 상태에 접근할 수 있다.                                                                                                                                                                                                                  |
| 설명 | 업싱크(upsync): 스포크 → 허브 방향으로 정보를 올려주는 것. Caspian이 각 스포크를 직접 보지 않아도 허브에서 모든 정보를 볼 수 있다.                                                                                                                                                                                                                                                               |
| 원문 | (3) In the hub cluster, Caspian runs periodically. At the beginning of each period, it gets the geo-location of the spoke clusters. (4) Then, it fetches carbon intensity data of those specific zones from Electricity Maps6, a tool that provides real-time information about the carbon intensity of electricity generation in various regions. |
| 해석 | (3) 허브 클러스터에서 Caspian은 주기적으로 실행된다. 각 주기 시작 시 스포크 클러스터의 지리적 위치를 가져온다. (4) 그런 다음 다양한 지역의 전기 발전 탄소집약도에 대한 실시간 정보를 제공하는 도구인 Electricity Maps(각 주6)에서 해당 지역의 탄소집약도 데이터를 가져온다.                                                                                                                                                                           |

|    |                                                                                                                                                                                                                                                                                                                                                                                                  |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 설명 | <p>각주6: <a href="https://app.electricitymaps.com/map">https://app.electricitymaps.com/map</a></p> <p>Electricity Maps는 전 세계 전력망의 실시간 탄소집약도를 지도로 보여주는 서비스다. Caspian은 이 서비스의 API를 통해 "지금 캐나다 전기가 얼마나 깨끗한가?"를 실시간으로 조회한다.</p>                                                                                                                                                                       |
| 원문 | (5) Carbon Tracker component in Caspian updates the carbon intensity field in ClusterInfo. (6) Next, Scheduler in Caspian gets the list of spoke clusters as well as AWs. Then, it calls the optimizer to find the best scheduling and placement for AWs.                                                                                                                                        |
| 해석 | <b>(5) Caspian의 Carbon Tracker 컴포넌트가 ClusterInfo의 탄소집약도 필드를 갱신한다. (6) 다음으로 Caspian의 Scheduler가 스포크 클러스터 목록 및 AW를 가져온다. 그런 다음 AW의 최적 스케줄링·배치를 찾기 위해 옵티마이저를 호출한다.</b>                                                                                                                                                                                                                              |
| 설명 | <p>5단계: Carbon Tracker가 방금 받은 탄소 데이터를 ClusterInfo에 써넣는다.</p> <p>6단계: Scheduler가 알고리즘(Algorithm 2, 3)을 돌려서 최적 계획을 세운다.</p>                                                                                                                                                                                                                                                                        |
| 원문 | (7) Finally, the scheduler updates targetCluster for those AWs that are scheduled during the upcoming time slot and removes the sustainability gates from the list of dispatchingGates. For those AWs that are not scheduled during the upcoming time slot, the scheduler adds a sustainability gate to the list of dispatchingGates.                                                            |
| 해석 | <b>(7) 마지막으로 스케줄러는 다음 시간 슬롯에 스케줄된 AW에 대해 targetCluster를 갱신하고 dispatchingGates 목록에서 sustainability gate를 제거한다. 다음 시간 슬롯에 스케줄되지 않은 AW에 대해서는 dispatchingGates 목록에 sustainability gate를 추가한다.</b>                                                                                                                                                                                                    |
| 설명 | <p>"이번 슬롯에 실행" → targetCluster 설정 + gate 제거 = "이제 가도 됩니다"</p> <p>"아직 대기" → gate 추가 = "아직 안 됩니다"</p>                                                                                                                                                                                                                                                                                              |
| 원문 | (8) MCAD Dispatcher is notified about these changes. Those AWs that are in Running or Dispatching phases, and now are requested by Caspian for suspension, will transit to Requeuing phase (to be queued again). Those AWs that are scheduled during the upcoming time slot, if they are ready to dispatch from MCAD perspective, the MCAD dispatcher changes their status to Dispatching phase. |
| 해석 | <b>(8) MCAD Dispatcher가 이 변경사항을 통보받는다. Running 또는 Dispatching 단계에 있다가 Caspian이 보류를 요청한 AW는 Requeuing 단계(다시 큐에 들어감)로 전환된다. 다음 시간 슬롯에 스케줄된 AW 중 MCAD 관점에서 디스패치 준비가 된 것은 MCAD Dispatcher가 상태를 Dispatching 단계로 변경한다.</b>                                                                                                                                                                             |
| 설명 | AW의 상태 전이: 대기 → 디스패칭 → 실행 중. 또는 실행 중 → 재큐잉(Caspian이 보류 요청 시).                                                                                                                                                                                                                                                                                                                                    |
| 원문 | (9) MCAD Syncer pushes the scheduled AWs to the corresponding spoke clusters. (10) MCAD Runner executes workloads in the spoke cluster and updates their status accordingly. (11) The status of AWs are pulled by the MCAD Syncer in the hub cluster, letting users aware of the status of their workloads.                                                                                      |
| 해석 | <b>(9) MCAD Syncer가 스케줄된 AW를 해당 스포크 클러스터로 밀어 넣는다. (10) MCAD Runner가 스포크 클러스터에서 워크로드를 실행하고 그에 따라 상태를 갱신한다. (11) AW의 상태가 허브 클러스터의 MCAD Syncer에 의해 풀(pull)되어, 사용자가 워크로드 상태를 알 수 있다.</b>                                                                                                                                                                                                             |
| 설명 | <p>9: 허브 → 스포크 방향으로 AW를 전송(push)</p> <p>10: 스포크에서 실제 GPU 작업 실행</p> <p>11: 스포크 상태를 다시 허브로 가져옴(pull) → 사용자가 "내 작업 지금 실행 중이야?"를 알 수 있다.</p>                                                                                                                                                                                                                                                         |

**명령 뿐만 아니라 받는 것도 하는구만**

## V. Experimental Analysis (실험 분석)

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>In this section, we compare the performance of Caspian for different types of problem instances. First, we investigate the impact of sustainability of clusters on the scheduling and placement decisions made by Caspian and compare the results with a baseline which does not consider sustainability of clusters in its decision making. We show that Caspian schedules a high percentage of workloads within their deadline while it also significantly reduces the carbon footprint due to the execution of workloads.</i> |
| 해석 | 이 섹션에서는 다양한 유형의 문제 인스턴스에 대한 Caspian의 성능을 비교한다. 먼저 클러스터의 지속가능성이 Caspian의 스케줄링·배치 결정에 미치는 영향을 조사하고, 클러스터의 지속가능성을 고려하지 않는 베이스라인과 결과를 비교한다. 우리는 Caspian이 워크로드 실행으로 인한 탄소발자국을 크게 줄이면서 높은 비율의 워크로드를 마감기한 내에 스케줄함을 보인다.                                                                                                                                                                                                                                                                                                                    |
| 설명 | <p style="text-align: center;"><b>이건 CAST도 했으니까</b></p> <p>실험 목적 1: "탄소를 고려한 Caspian"과 "탄소를 무시한 기존 방식"을 비교해서 탄소 절감 효과를 증명.</p> <p>실험 목적 2: <math>\omega_1</math>(탄소 가중치)을 여러 값으로 바꿔보며 "탄소↑ vs 마감 위반↑" 트레이드오프 분석.</p> <p style="text-align: right;"><b>이거는 soft제약조건에 대한 실험</b></p>                                                                                                                                                                                                                                                   |
| 원문 | <i>Then, we vary the sustainability weight (<math>\omega_1</math>) and investigate its impact on lateness, carbon emission and completion time of workloads. We show that by increasing the sustainability weight, a higher amount of carbon is reduced; but a higher percentage of workloads may miss their deadline. We conclude that great care has to be taken when setting the values of the weights <math>\omega_k</math>.</i>                                                                                                |
| 해석 | 그런 다음 지속가능성 가중치 $\omega_1$ 을 변화시키고 지연, 탄소 배출, 워크로드 완료 시간에 미치는 영향을 조사한다. 우리는 지속가능성 가중치를 높이면 더 많은 탄소가 절감되지만, 더 높은 비율의 워크로드가 마감기한을 놓칠 수 있음을 보인다. 우리는 가중치 $\omega_k$ 값을 설정할 때 매우 신중해야 한다는 결론을 내린다.                                                                                                                                                                                                                                                                                                                                      |
| 설명 | 핵심 결론 미리보기: $\omega_1$ 을 너무 높이면(탄소 절감에 너무 집착하면) 작업이 너무 늦어져서 마감을 못 지킨다. 균형이 중요!                                                                                                                                                                                                                                                                                                                                                                                                                                                      |

## A. Experimental Setup (실험 설정)

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>In conducting our experiments, we consider a configuration consisting of four Kubernetes clusters: a hub cluster and three spoke clusters that are denoted by Spoke1, Spoke2, and Spoke3. All clusters are created using k3d7. Each spoke cluster consists of two homogeneous nodes, each equipped with 16 GPUs. Thus, each spoke cluster has a total of 32 GPU cores.</i>                                                                                                                                                                                                                                |
| 해석 | 실험에서는 허브 클러스터 1개와 Spoke1, Spoke2, Spoke3로 표기되는 스포크 클러스터 3개, 총 4개의 쿠버네티스 클러스터로 구성된 환경을 고려한다. 모든 클러스터는 k3d를 사용해 생성된다(각주7). 각 스포크 클러스터는 16개 GPU를 장착한 동질적인 노드 2개로 구성되므로, 각 스포크 클러스터의 총 GPU 코어는 32개다.                                                                                                                                                                                                                                                                                                                                                                                                             |
| 설명 | <p>각주7: <a href="https://k3d.io">https://k3d.io</a> — Docker 위에서 가벼운 쿠버네티스를 만드는 도구. 실제 물리 서버 없이 노트북에서도 멀티클러스터 환경을 만들 수 있다.</p> <p>즉 실험을 위한 가상의 서버들을 만든 것이다.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 원문 | <i>We assume that spoke clusters are located in three different electricity grid regions. Spoke1 is located in Ontario, Canada (CA-ON), Spoke2 in Kansai, Japan (JP-KN), and Spoke3 in Germany (DE). Figure 3 shows the carbon intensity of electricity source of each cluster during the 40-hour period starting from November 14th, 2023. We employ Electricity Map to get carbon intensity data for the grid regions. The data indicates that electricity source of Spoke1 has the lowest carbon intensity, whereas electricity sources of Spoke2 and Spoke3 both illustrate high carbon intensities.</i> |
| 해석 | 스포크 클러스터들이 세 가지 다른 전력망 지역에 위치한다고 가정한다. Spoke1은 캐나다 온타리오(CA-ON), Spoke2는 일본 간사이(JP-KN), Spoke3는 독일(DE)에 위치한다. Figure 3은 2023년 11월 14일을 시작으로 40시간 동안 각 클러스터의 탄소집약도를 보여준다. 전력망 지역의 탄소집약도 데이터를 얻기 위해 Electricity Map을 사용한다. 데이터는 Spoke1의 전력원이 가장 낮은 탄소집약도를 가지며, Spoke2와 Spoke3의 전력원은 모두 높은 탄소 집약도를 나타냄을 보여준다.                                                                                                                                                                                                                                                                                                    |
| 설명 | <p>실제 2023년 11월 14일 데이터를 사용했다.</p> <p>Spoke1(캐나다): 수력발전이 많아서 항상 깨끗. 탄소집약도 약 30 gCO<sub>2</sub>/kWh.</p> <p>Spoke2(일본), Spoke3(독일): 화석연료가 많아서 더럽다. 탄소집약도 400~520 gCO<sub>2</sub>/kWh. 캐나다보다 10~15배!</p>                                                                                                                                                                                                                                                                                                                                                                                                       |

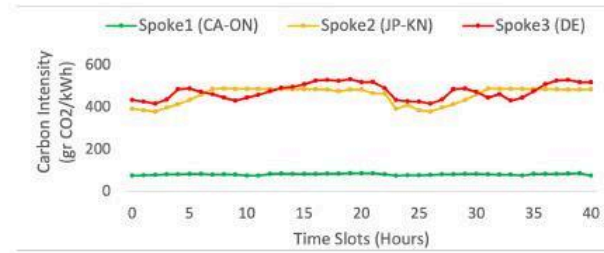


Fig. 3: Hourly carbon intensity of clusters.

by Caspian for suspension, will transit to Requeuing phase

Fig. 3: Hourly carbon intensity (gCO<sub>2</sub>/kWh) — Spoke1(캐나다, 녹색)은 항상 낮고, Spoke2(일본, 주황)·Spoke3(독일, 빨강)는 400~520 범위.

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | We assume that Spoke2 is twice more power efficient than other spoke clusters and set $p_1 = p_3 = 200$ , $p_2 = 100$ . This implies that although Spoke2 and Spoke3 both are powered by a high carbon intensity electricity source; Spoke2 is more power efficient and is expected to be more preferable by Caspian.                                                                                                                                                                                                                                                                                                                                                                                        |
| 해석 | <b>Spoke2가 다른 스포크보다 전력 효율이 2배 더 좋다고 가정하여 <math>p_1 = p_3 = 200</math>, <math>p_2 = 100</math>으로 설정한다. 이는 Spoke2와 Spoke3가 모두 탄소집약도가 높은 전력원을 사용하지만, Spoke2가 더 전력 효율적이라서 Caspian에게 더 선호될 것임을 의미한다.</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 설명 | 원문: "We assume that Spoke2 is twice more power efficient than other spoke clusters and set $p_1 = p_3 = 200$ , $p_2 = 100$ ." — 정확한 수치: $p_1=p_3=200$ , $p_2=100$ .<br>의도적인 "함정" 실험 설계다.<br>일본(Spoke2)과 독일(Spoke3): 둘 다 전기가 더럽다. 근데 일본은 서버 자체가 효율적이라 같은 작업에 전기를 절반만 쓴다.<br>→ Caspian이 탄소 절감 = 전기 더러움 × 전기 사용량을 제대로 계산하는지 테스트.<br>→ 올바른 Caspian: 일본 > 독일 순으로 선호. (더럽긴 해도 절반만 쓰니까)                                                                                                                                                                                                                                                                                                                             |
| 원문 | In our experimental setup, we focus on long-running workloads with execution time ranging from one hour to five hours, e.g. AI training jobs. We exclude short-running workloads from our experiments as they typically come with tight deadlines and cannot tolerate delays.                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 해석 | <b>실험 설정에서 우리는 1~5시간 실행 시간을 가진 장시간 실행 워크로드(예: AI 학습 잡)에 집중한다. 단시간 실행 워크로드는 주로 마감기한이 빡빡하고 지연을 허용하지 않으므로 실험에서 제외한다.</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 설명 | 단시간 작업(예: 웹 요청 처리, 실시간 서빙)은 "지금 당장" 해야 해서 탄소를 고려해 미루는 게 불가능하다. 그래서 "미뤄도 되는" 장시간 배치 작업만 실험한다.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 원문 | We use Poisson distribution for the arrival of 200 workloads over 24 hours. Each workload is represented by an AW consisting of a BusyBox8 image to execute the workload's task. The task assigned to each AW is a simple UNIX sleep command9.                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 해석 | <b>24시간 동안 200개 워크로드의 도착에 포아송 분포를 사용한다. 각 워크로드는 워크로드의 작업을 실행하기 위한 BusyBox 이미지로 구성된 AW로 표현된다(각주8). 각 AW에 할당된 작업은 단순한 UNIX sleep 명령이다(각주9).</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 설명 | 각주8: <a href="https://busybox.net">https://busybox.net</a> — 아주 가벼운 리눅스 도구 모음<br>각주9: 실험에서는 실제로 자원 사용률을 직접 측정하지 않고 자원 요구량에 비례한다고 가정한다.<br><b>Poisson 분포:</b> 버스처럼 한꺼번에 몰려오기도 하고 드문드문 오기도 하는 랜덤 도착 패턴.<br><b>sleep 명령:</b> 실제 AI 학습 대신 단순히 "n초 동안 멈춰있어라"를 사용. 이러면 GPU를 실제로 쓰진 않지만 자원을 예약해놓는다. 시스템이 제대로 작동하는지 확인하기엔 충분.                                                                                                                                                                                                                                                                                                                                                                                        |
| 원문 | To perform an extensive experiments, we consider various characteristics of the workloads. The GPU requirement of each workload is randomly drawn from the uniform distribution $U(1, 5)$ . Similarly, the run time for each workload is randomly chosen from another independent uniform distribution, $U(1, 5)$ . We assume that workloads have deadlines proportional to their run times. Thus, the deadline is calculated based on a slowdown factor $\beta$ . For example, for workload $i$ , if the workload is submitted in time slot $t$ , the deadline of workload is defined as $d_i = t + \beta \times l_i$ , where the value of $\beta$ is randomly chosen from uniform distribution $U(2, 4)$ . |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 해석 | 광범위한 실험을 위해 다양한 워크로드 특성을 고려한다. 각 워크로드의 GPU 요구량은 균일분포 $U(1,5)$ 에서 무작위로 추출된다. 마찬가지로 각 워크로드의 실행 시간은 독립 균일분포 $U(1,5)$ 에서 무작위로 선택된다. 워크로드의 마감기한은 실행 시간에 비례한다고 가정한다. 따라서 마감기한은 슬로다운 인자 $\beta$ 를 기반으로 계산된다. 예를 들어 워크로드 $i$ 가 시간 슬롯 $t$ 에 제출되면, 마감기한은 $d_i = t + \beta \times l_i$ 로 정의되며, $\beta$ 값은 균일분포 $U(2,4)$ 에서 무작위로 선택된다.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 설명 | <p>각 작업의 특성을 랜덤으로 생성해서 다양한 상황을 시뮬레이션:</p> <ul style="list-style-type: none"> <li>GPU 요구량: 1~5개 중 랜덤</li> <li>실행 시간 <math>l_i</math>: 1~5 시간 중 랜덤 (균일분포 <math>U(1,5)</math> 사용)</li> <li>마감기한: <math>d_i = t + \beta \times l_i</math>, <math>\beta \sim U(2,4)</math> (균일분포 <math>U(2,4)</math>에서 선택)</li> </ul> <p>원문: "the deadline is calculated based on a slowdown factor <math>\beta</math>. For example, for workload <math>i</math>, if the workload is submitted in time slot <math>t</math>, the deadline of workload is defined as <math>d_i = t + \beta \times l_i</math>, where the value of <math>\beta</math> is randomly chosen from uniform distribution <math>U(2, 4)</math>."</p> <p>예: 3시간짜리 작업, <math>\beta=3</math>이면 마감 = 제출시각 + 9시간. 즉 9시간의 여유.</p> <p><math>\beta</math>가 클수록 마감 여유가 많다 = 더 유연하게 스케줄 가능.</p> |

## B. Analysis of Results (결과 분석)

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>First, we investigate the impact of sustainability of clusters on the scheduling and placement decisions. We run Caspian in two different modes but under the same environmental conditions: (1) sustainable mode in which Caspian considers a non-zero weight for sustainability objective, i.e., <math>\omega_1 = 1</math>, and (2) baseline mode in which the weight for sustainability objective is zero, <math>\omega_1 = 0</math>. For both modes, we consider the same preference for the other objective functions, namely <math>\omega_2 = \omega_3 = 1</math>.</i> |
| 해석 | 먼저 클러스터의 지속가능성이 스케줄링·배치 결정에 미치는 영향을 조사한다. 우리는 동일한 환경 조건에서 Caspian을 두 가지 다른 모드로 실행한다: (1) 지속가능성 목적에 비영 가중치를 고려하는 지속가능 모드( $\omega_1=1$ ), (2) 지속가능성 목적 가중치가 0인 베이스라인 모드( $\omega_1=0$ ). 두 모드 모두 나머지 목적함수에는 동일한 선호도 $\omega_2=\omega_3=1$ 을 고려한다.                                                                                                                                                                                                                                                                                                                                |
| 설명 | <p>원문: "For both modes, we consider the same preference for the other objective functions, namely <math>\omega_2 = \omega_3 = 1</math>." — 두 모드 모두 <math>\omega_2=\omega_3=1</math>로 동일하게 설정.</p> <p>지속가능 모드(<math>\omega_1=1</math>): 탄소도 신경 씬.</p> <p>베이스라인 모드(<math>\omega_1=0</math>): 탄소는 무시하고 마감·완료시간만 최적화. 기존 스케줄러와 동일.</p>                                                                                                                                                                                                                                              |
| 원문 | <i>Figure 4a illustrates the GPU allocation of clusters over time slots when Caspian is in the sustainable mode. We observe that Caspian first starts allocating workloads to Spoke1 which has the lowest carbon emission rate. With the increase of the load over time, Caspian starts using Spoke2 (at time slot <math>t = 5</math>) to manage QoS of workloads. Later, it also uses Spoke3 (time slot <math>t = 9</math>) to schedule workloads. Then, when the peak time is passed, Caspian reduces the allocations on Spoke2 and Spoke3 and relies more on Spoke1.</i>     |
| 해석 | Figure 4a는 Caspian이 지속가능 모드일 때 시간 슬롯에 걸친 클러스터의 GPU 할당을 나타낸다. 우리는 Caspian이 탄소 배출률이 가장 낮은 Spoke1에 먼저 워크로드를 배정하기 시작함을 관찰한다. 시간이 지나면서 부하가 증가함에 따라, Caspian은 워크로드의 QoS를 관리하기 위해 시간 슬롯 $t=5$ 에서 Spoke2를 사용하기 시작한다. 이후 시간 슬롯 $t=9$ 에서 Spoke3도 사용한다. 그런 다음 피크 시간이 지나면 Caspian은 Spoke2와 Spoke3의 할당을 줄이고 Spoke1에 더 의존한다.                                                                                                                                                                                                                                                                  |
| 설명 | <p>Caspian의 행동을 단계로 설명:</p> <ol style="list-style-type: none"> <li><math>t=1\sim4</math>: Spoke1(캐나다)만 사용. 깨끗한 전기니까.</li> <li><math>t=5</math>: 작업이 너무 많아져서 Spoke1만으론 부족. 마감을 지키려면 Spoke2(일본)도 써야 한다.</li> <li><math>t=9</math>: 더 바빠져서 Spoke3(독일)도 동원.</li> <li>피크가 지나면: 다시 Spoke1으로 집중.</li> </ol> <p>→ Caspian이 "깨끗한 곳 우선, 마감 위협받을 때만 더러운 곳"이라는 전략을 자동으로 구사한다!</p>                                                                                                                                                                                                           |
| 원문 | <i>Thus, in all, Spoke1 has the highest GPU allocation compared to the other spoke clusters. We also observe that although the carbon intensity of electricity source for Spoke2 and Spoke3 is in the same range, Spoke2 receives higher workloads requests compared to Spoke3. The reason is that Spoke3 has a lower power efficiency, hence not preferred by Caspian. Another observation from the figure is the makespan of execution of workloads. We observe that all workloads are completed by time slot <math>t = 39</math>.</i>                                        |

|    |                                                                                                                                                                                                                                                             |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 해석 | 따라서 전체적으로 Spoke1이 다른 스포크 클러스터들에 비해 가장 높은 GPU 할당량을 가진다. 또한 Spoke2와 Spoke3의 탄소집약도가 동일한 범위에 있음에도 Spoke2가 Spoke3보다 더 많은 워크로드 요청을 받음을 관찰한다. 그 이유는 Spoke3의 전력 효율이 낮아 Caspian이 선호하지 않기 때문이다. 또 다른 관찰은 워크로드 실행의 makespan이다. 우리는 모든 워크로드가 시간 슬롯 $t=39$ 까지 완료됨을 관찰한다. |
| 설명 | Spoke2(일본) > Spoke3(독일) 사용: 둘 다 전기가 더럽지만, 일본 서버가 전기를 절반만 쓰니까 일본이 낫다. Caspian이 이걸 자동으로 계산해서 올바른 선택을 했다!<br>Makespan = 39시간: 모든 200개 작업이 39시간 안에 완료됨.                                                                                                         |

|    |                                                                                                                                                                                                                                                                                                             |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | Figure 4b illustrates the GPU allocation of clusters when Caspian is in the baseline mode. In this mode, a lower makespan is achieved (29 hours). We also observe that Caspian distributes workloads equally among clusters to guarantee low average completion time and low average lateness of workloads. |
| 해석 | Figure 4b는 Caspian이 베이스라인 모드일 때 클러스터의 GPU 할당을 나타낸다. 이 모드에서 더 낮은 makespan이 달성된다(29시간). 또한 Caspian이 평균 완료 시간과 평균 지연을 낮게 유지하기 위해 클러스터 간에 워크로드를 균등하게 배분함을 관찰한다.                                                                                                                                                 |
| 설명 | 베이스라인은 "빠르기만" 하다: 29시간에 끝내지만 탄소를 더 많이 냄.<br>지속가능 모드는 "탄소를 아끼느라" 39시간 걸림. 10시간의 여유를 탄소 절감에 사용한 것.                                                                                                                                                                                                            |

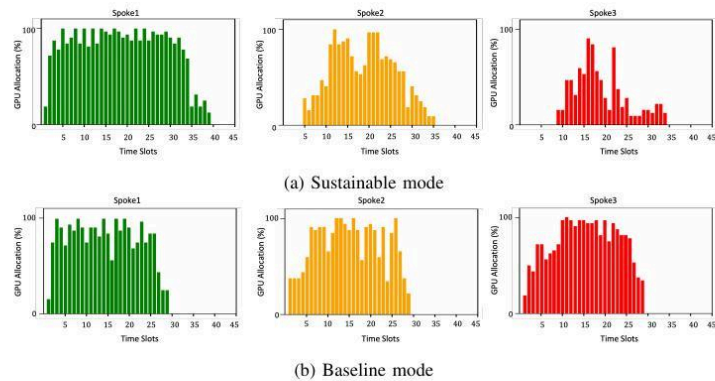


Fig. 4: GPU Allocation of clusters over time slots running Caspian in (a) sustainable mode; (b) baseline mode.

the baseline mode. In this mode, a lower makespan is achieved (29 hours). We also observe that Caspian distributes workloads equally among clusters to guarantee low average completion time and low average lateness of workloads.

Figure 5 compares the overall performance of Caspian in the two modes. Figure 5a illustrates the carbon measure

the value of  $\alpha$  when Caspian is in baseline mode. Since in this mode, the scheduler does not postpone any workload due to sustainability, all workloads complete by their deadline.

In the next set of experiments, we investigate the impact of weight  $\omega_1$  on overall performance. We set  $\omega_2 = \omega_3 = 1$ , and vary the value of  $\omega_1$  from zero to two, assigning various

Fig. 4: GPU Allocation — (a) 지속가능 모드: Spoke1 우선 사용, Makespan=39h. (b) 베이스라인: 균등 분배, Makespan=29h.

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | Figure 5 compares the overall performance of Caspian in the two modes. Figure 5a illustrates the carbon measure of execution of workloads in each mode. Carbon measure in time slot $t$ reflects the total carbon footprint generated by the execution of workloads from the first time slot to the current time slot $t$ . We observe that when Caspian is in sustainable mode, a significantly lower carbon emission is achieved compared to the baseline mode. |
| 해석 | Figure 5는 두 모드에서 Caspian의 전반적인 성능을 비교한다. Figure 5a는 각 모드에서 워크로드 실행의 탄소 척도를 나타낸다. 시간 슬롯 $t$ 에서의 탄소 척도는 첫 번째 시간 슬롯부터 현재 시간 슬롯 $t$ 까지 워크로드 실행으로 발생한 총 탄소발자국을 반영한다. 우리는 Caspian이 지속가능 모드일 때 베이스라인 모드에 비해 탄소 배출이 크게 낮아짐을 관찰한다.                                                                                                                                                                                                                                         |
| 설명 | 그래프를 보면 지속가능 모드의 선이 베이스라인보다 훨씬 낮다. 탄소를 덜 내고 있다는 뜻.                                                                                                                                                                                                                                                                                                                                                                                                                |
| 원문 | We define a performance metric carbon saving (CS) as follows, $CS = 100 \times (C - C^*) / C$ , where $C$ and $C^*$ are the carbon measures in baseline and sustainable modes, respectively. In this experiment, we observe that the value of CS is 38.2% indicating that by executing workloads in sustainable mode, Caspian reduces carbon emission by 38.2%.                                                                                                   |
| 해석 | 우리는 성능 지표인 탄소 절감(CS)을 다음과 같이 정의한다: $CS = 100 \times (C - C^*) / C$ . 여기서 $C$ 와 $C^*$ 는 각각 베이스라인 모드와 지속가능 모드에서의 탄소 척도다. 이 실험에서 $CS = 38.2\%$ 로, 지속가능 모드에서 워크로드를 실행함으로써 Caspian이 탄소 배출을 38.2% 줄인다는 것을 나타낸다.                                                                                                                                                                                                                                                         |

|    |                                                                                       |
|----|---------------------------------------------------------------------------------------|
| 설명 | 이 38.2%는 $\omega_1=1$ 설정. 초록에서 언급한 "33%"는 $\omega_1=0.5$ 설정이다. 같은 논문에서 두 수치가 나오는 이유다. |
|----|---------------------------------------------------------------------------------------|

▶ 탄소 절감률(CS) 정의 — 식 (9)

the baseline mode. In this mode, a lower makespan is achieved (29 hours). We also observe that Caspian distributes workloads equally among clusters to guarantee low average completion time and low average lateness of workloads.

$$\text{식(9): } CS = 100 \times (C - C^*) / C$$

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>To evaluate the efficiency of Caspian regarding the QoS of workloads, we define <math>\alpha</math> as the completion time ratio of workloads. The higher value of <math>\alpha</math> indicates the longer response time of workloads compared to their deadline. Let us denote the submission time of workload <math>i</math> by <math>s_i</math> and the completion time by <math>f_i</math>, the completion time ratio of workload <math>i</math> is given by <math>\alpha = (f_i - s_i)/(d_i - s_i)</math>. Here, <math>\alpha \leq 1</math> indicates that the workload is completed by its deadline. The greater values of <math>\alpha</math> indicate the lateness of workloads.</i>                                                                                                           |
| 해석 | 워크로드의 QoS에 관한 Caspian의 효율성을 평가하기 위해, $\alpha$ 를 워크로드의 완료 시간 비율로 정의한다. $\alpha$ 값이 클 수록 마감기한 대비 응답 시간이 더 길다는 것을 나타낸다. 워크로드 $i$ 의 제출 시각을 $s_i$ , 완료 시각을 $f_i$ 로 표기하면, 완료 시간 비율은 $\alpha = (f_i - s_i)/(d_i - s_i)$ 로 주어진다. 여기서 $\alpha \leq 1$ 은 워크로드가 마감기한 내에 완료됨을 나타낸다. $\alpha$ 값이 클수록 지연이 심함을 나타낸다.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 설명 | 원문: "we define $\alpha$ as the completion time ratio of workloads. The higher value of $\alpha$ indicates the longer response time of workloads compared to their deadline. The completion time ratio of workload $i$ is given by $\alpha = (f_i - s_i)/(d_i - s_i)$ . Here, $\alpha \leq 1$ indicates that the workload is completed by its deadline. The greater values of $\alpha$ indicate the lateness of workloads."<br>$\alpha$ 값 해석 예시: <ul style="list-style-type: none"> <li>• 제출: 오전 9시, 마감: 오전 9시 + 10시간 = 오후 7시, 완료: 오후 1시(+4시간)</li> <li>• <math>\alpha = (4시간) / (10시간) = 0.4 \rightarrow</math> 마감의 40% 지점에서 완료. 여유롭게 성공.</li> <li>• <math>\alpha = 0.6 \rightarrow</math> 마감의 60% 지점에서 완료 (이 논문의 평균)</li> <li>• <math>\alpha = 1.2 \rightarrow</math> 마감을 20% 초과해서 완료 = 마감 위반!</li> </ul> |

▶ 완료시간 비율( $\alpha$ ) 정의

performance metric *carbon saving* (CS) as follows,

$$CS = 100 \times \frac{C - C^*}{C}, \tag{9}$$

where  $C$  and  $C^*$  are the carbon measures in baseline and sustainable modes, respectively. In this experiment, we observe

$$\alpha = (f_i - s_i) / (d_i - s_i) \rightarrow \alpha \leq 1 \text{이면 마감 준수, } \alpha > 1 \text{이면 마감 위반}$$

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>Figure 5b illustrates a histogram of the value of <math>\alpha</math> for workloads scheduled by Caspian in sustainable mode. We observe that a relatively high percentage of workloads are executed by their deadline and just a fraction of workloads (around 7.2%) miss their deadline. The reason is that, in sustainable mode, Caspian tries to balance a trade off between all objective functions. Thus, to reduce the carbon emission, some workloads may miss their deadline. However, since Caspian also considers QoS, this percentage is pretty low.</i> |
| 해석 | Figure 5b는 지속가능 모드에서 Caspian이 스케줄한 워크로드의 $\alpha$ 값 히스토그램을 나타낸다. 우리는 상당히 높은 비율의 워크로드가 마감기한 내에 실행되고 일부 워크로드(약 7.2%)만 마감기한을 놓침을 관찰한다. 그 이유는 지속 가능 모드에서 Caspian이 모든 목적함수 간의 트레이드오프를 균형 잡으려 하기 때문이다. 따라서 탄소 배출을 줄이기 위해 일부 워크로드가 마감기한을 놓칠 수 있다. 그러나 Caspian이 QoS도 고려하므로 이 비율은 꽤 낮다.                                                                                                                                                                                                                                                                                          |
| 설명 | $\omega_1=1$ 에서 7.2%는 마감 위반. 100개 중 7개 정도가 늦음. 탄소를 38.2% 줄이는 대가로 7.2%가 늦는 것이다.<br>이게 나쁜가? $\rightarrow$ 맥락에 따라 다르다. AI 학습 잡은 조금 늦어도 괜찮은 경우가 많다.                                                                                                                                                                                                                                                                                                                                                                                                                         |

|    |                                                                                                                                                                                                                                                                                    |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>Figure 5c shows the histogram of the value of <math>\alpha</math> when Caspian is in baseline mode. Since in this mode, the scheduler does not postpone any workload due to sustainability, all workloads complete by their deadline.</i>                                       |
| 해석 | Figure 5c는 Caspian이 베이스라인 모드일 때 $\alpha$ 값의 히스토그램을 보여준다. 이 모드에서는 스케줄러가 지속가능성 때문에 어떤 워크로드도 연기하지 않으므로, 모든 워크로드가 마감기한 내에 완료된다.                                                                                                                                                        |
| 설명 | 원문: "Figure 5c shows the histogram of the value of $\alpha$ when Caspian is in baseline mode. Since in this mode, the scheduler does not postpone any workload due to sustainability, all workloads complete by their deadline."<br>베이스라인: 마감 위반 0%. 탄소를 안 고려하니까 탄소가 나와도 빠르게 처리한다. |

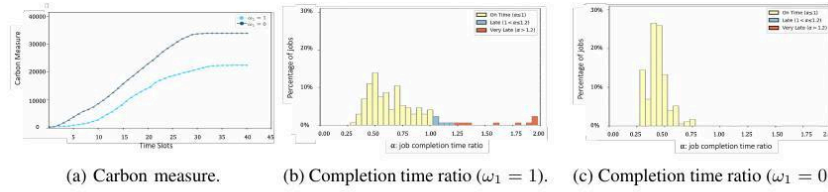


Fig. 5: Overall performance of Caspian: sustainable mode vs baseline mode.

Fig. 5: (a) 누적 탄소 배출: 지속가능 모드가 훨씬 낮다. (b)  $\omega_1=1$   $\alpha$  분포: 대부분 마감 준수, 7.2% 위반. (c)  $\omega_1=0$   $\alpha$  분포: 모두 마감 준수.

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>In the next set of experiments, we investigate the impact of weight <math>\omega_1</math> on overall performance. We set <math>\omega_2 = \omega_3 = 1</math>, and vary the value of <math>\omega_1</math> from zero to two, assigning various weights to the sustainability objective. Here, <math>\omega_1 = 0</math> is the baseline case where Caspian does not incorporate sustainability aspects of clusters in its decision making. With the increase in the value of <math>\omega_1</math>, Caspian considers a higher weight for sustainability.</i>                                              |
| 해석 | 다음 실험 세트에서는 가중치 $\omega_1$ 이 전반적인 성능에 미치는 영향을 조사한다. $\omega_2=\omega_3=1$ 로 설정하고, $\omega_1$ 값을 0에서 2까지 변화시키며 지속가능성 목적에 다양한 가중치를 부여한다. 여기서 $\omega_1=0$ 은 Caspian이 클러스터의 지속가능성 측면을 의사결정에 포함하지 않는 베이스라인 케이스다. $\omega_1$ 값이 증가할수록 Caspian은 지속가능성에 더 높은 가중치를 고려한다.                                                                                                                                                                                                                                                                                                                                              |
| 설명 | 원문: "In the next set of experiments, we investigate the impact of weight $\omega_1$ on overall performance. We set $\omega_2 = \omega_3 = 1$ , and vary the value of $\omega_1$ from zero to two, assigning various weights to the sustainability objective. Here, $\omega_1 = 0$ is the baseline case where Caspian does not incorporate sustainability aspects of clusters in its decision making. With the increase in the value of $\omega_1$ , Caspian considers a higher weight for sustainability."<br>실험: $\omega_1$ 을 0, 0.25, 0.5, 1, 2로 바꿔보며 탄소 절감 vs 마감 위반의 트레이드오프를 측정.                           |
| 원문 | <i>Figure 6a shows the percentage of carbon saving obtained due to various values of <math>\omega_1</math>. The value of carbon saving is obtained based on Equation (9), in which <math>C</math> is the carbon measure using Caspian in the baseline mode and <math>C^*</math> is the carbon measure when we run Caspian with the given value of <math>\omega_1</math>. We observe that by increasing <math>\omega_1</math>, a higher value of carbon saving is achieved. For example, for <math>\omega_1 = 0.25</math>, carbon saving is 19.4% while for <math>\omega_1 = 2</math>, the value is 42.4%.</i> |
| 해석 | Figure 6a는 $\omega_1$ 의 다양한 값에 따른 탄소 절감률을 보여준다. 탄소 절감 값은 식(9)에 기반해 구하며, $C$ 는 베이스라인 모드 Caspian의 탄소 척도이고 $C^*$ 는 주어진 $\omega_1$ 값으로 Caspian을 실행할 때의 탄소 척도다. 우리는 $\omega_1$ 을 높일수록 더 높은 탄소 절감 값이 달성됨을 관찰한다. 예를 들어 $\omega_1=0.25$ 이면 탄소 절감은 19.4%이고, $\omega_1=2$ 이면 42.4%다.                                                                                                                                                                                                                                                                                                                                    |
| 설명 | 원문: "We observe that by increasing $\omega_1$ , a higher value of carbon saving is achieved. For example, for $\omega_1 = 0.25$ , carbon saving is 19.4% while for $\omega_1 = 2$ , the value is 42.4%."<br>$\omega_1$ 이 높을수록 탄소를 더 줄인다. 당연한 결과. 그런데...                                                                                                                                                                                                                                                                                                                                                       |
| 원문 | <i>In Figure 6b, the average completion time ratio of all workloads for different values of <math>\omega_1</math> is depicted. We observe that as the value of <math>\omega_1</math> increases, more emphasis is placed on the first objective <math>Z_1</math>, leading to an increase in the completion time of workloads. For instance, for <math>\omega_1 = 0</math>, the average completion ratio is 0.49, while for <math>\omega_1 = 1</math>, this ratio is 0.72.</i>                                                                                                                                  |
| 해석 | Figure 6b에는 $\omega_1$ 의 다양한 값에 따른 모든 워크로드의 평균 완료 시간 비율이 묘사되어 있다. 우리는 $\omega_1$ 값이 증가할수록 첫 번째 목적 $Z_1$ 에 더 많은 강조가 주어져 워크로드의 완료 시간이 증가함을 관찰한다. 예를 들어 $\omega_1=0$ 이면 평균 완료 비율은 0.49이고, $\omega_1=1$ 이면 이 비율은 0.72다.                                                                                                                                                                                                                                                                                                                                                                                           |
| 설명 | 원문: "We observe that as the value of $\omega_1$ increases, more emphasis is placed on the first objective $Z_1$ , leading to an increase in the completion time of workloads. For instance, for $\omega_1 = 0$ , the average completion ratio is 0.49, while for $\omega_1 = 1$ , this ratio is 0.72."<br>$\omega_1 \uparrow \rightarrow$ 작업을 더 미룬다 $\rightarrow \alpha$ 도 올라간다 (더 늦게 끝남). $\omega_1=0$ 이면 평균 $\alpha=0.49$ , $\omega_1=1$ 이면 $\alpha=0.72$ .                                                                                                                                                 |
| 원문 | <i>We also observe a big jump in completion time ratio over the case of <math>\omega_1 = 1</math> and the case of <math>\omega_1 = 2</math>. This jump is also observed in Figure 6c where the percentage of workloads that meet their deadline is plotted. We observe that for <math>\omega_1 = 2</math>, about 15% of workloads miss their deadline, which is not a negligible value.</i>                                                                                                                                                                                                                   |
| 해석 | 또한 우리는 $\omega_1=1$ 과 $\omega_1=2$ 사이에서 완료 시간 비율이 크게 점프함을 관찰한다. 이 점프는 마감기한을 지키는 워크로드의 비율을 나타낸 Figure 6c에서도 관찰된다. 우리는 $\omega_1=2$ 에서 약 15%의 워크로드가 마감기한을 놓침을 관찰하며, 이는 무시할 수 없는 값이다.                                                                                                                                                                                                                                                                                                                                                                                                                            |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 설명 | <p>원문: "We also observe a big jump in completion time ratio over the case of <math>\omega_1 = 1</math> and the case of <math>\omega_1 = 2</math>. This jump is also observed in Figure 6c where the percentage of workloads that meet their deadline is plotted. We observe that for <math>\omega_1 = 2</math>, about 15% of workloads miss their deadline, which is not a negligible value."</p> <p>⚠ 위험 경고: <math>\omega_1=1</math>에서 2로 올리면 마감 위반이 7.2%에서 15%로 <b>2배 급등!</b></p> <p><math>\omega_1</math>을 "조금만 더" 올렸는데 결과가 비선형적으로 나빠진다. 운영자가 꼭 주의해야 할 부분.</p> |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | For $\omega_1 = 0.5$ , we get 33.21% carbon reduction, 98.28% meeting deadline, and 0.6 completion ratio. This experiment demonstrates the importance of setting the right values for the preference factors.                                                                                                                                                                                                                                                                                                                               |
| 해석 | <b><math>\omega_1=0.5</math>에서 탄소 절감 33.21%, 마감 준수 98.28%, 완료 비율 0.6을 얻는다. 이 실험은 선호 인자의 올바른 값 설정의 중요성을 보여준다.</b>                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 설명 | <p>원문: "For <math>\omega_1 = 0.5</math>, we get 33.21% carbon reduction, 98.28% meeting deadline, and 0.6 completion ratio. This experiment demonstrates the importance of setting the right values for the preference factors."</p> <p><b><math>\omega_1=0.5</math>가 권장 설정인 이유:</b></p> <ul style="list-style-type: none"> <li>탄소 33% 절감: 의미 있는 환경 효과</li> <li>마감 준수 98.28%: 100개 중 2개 미만만 늦음. 실용적으로 충분.</li> <li>완료 비율 0.6: 마감의 60%에서 완료. 여유 있게 끝남.</li> </ul> <p>이 세 가지를 함께 보면 <math>\omega_1=0.5</math>가 탄소와 QoS를 가장 균형 있게 잡는 설정이다.</p> |

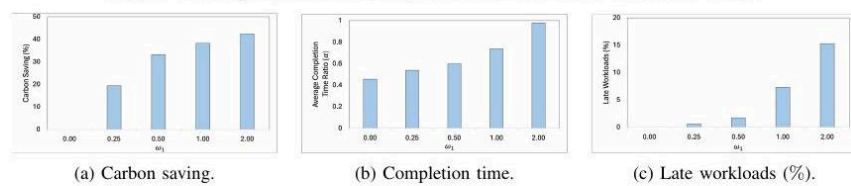


Fig. 6: Performance of Caspian for various values of  $\omega_1$ .

of workloads, while satisfying Quality of Service (QoS) requirements. We designed a framework for multi-cluster environments in which Caspian interacts with the multi-cluster management platform. Caspian periodically collects informa-

- [6] X. Fan, W.-D. Weber, and L. A. Barroso. Power provisioning for a warehouse-sized computer. *ACM SIGARCH Computer Architecture News*, 35(2):13–23, 2007.
- [7] H. Ferreboeuf, F. Berthoud, P. Bihouix, P. Fabre, D. Kaplan, L. Lefèvre, et al. Lean ict: Towards digital sobriety. *Report for the Think Tank The*

Fig. 6:  $\omega_1$ 별 성능 — (a) 탄소 절감을 ↑, (b) 평균 완료비율 ↑, (c) 마감 위반율:  $\omega_1=1 \rightarrow 2$  구간에서 급등.

## VI. Conclusion and Future Work (결론 및 향후 연구)

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원문 | <i>In this paper, we proposed Caspian, a carbon-aware batch scheduler in multi-cluster Kubernetes environments, with the aim of minimizing carbon emission due to the execution of workloads, while satisfying Quality of Service (QoS) requirements.</i>                                                                                                                                                                                                                                                |
| 해석 | 이 논문에서 우리는 QoS 요구사항을 만족하면서 워크로드 실행으로 인한 탄소 배출을 최소화하는 것을 목표로 하는, 멀티클러스터 쿠버네티스 환경의 탄소인지 배치 스케줄러 <b>Caspian</b> 을 제안했다.                                                                                                                                                                                                                                                                                                                                                                                     |
| 설명 | "우리가 해결한 문제" 요약: 여러 나라 서버에서 AI 학습 같은 작업을 돌릴 때, 마감은 지키면서 CO <sub>2</sub> 를 최대한 줄이는 시스템을 만들었다.                                                                                                                                                                                                                                                                                                                                                                                                             |
| 원문 | <i>We designed a framework for multi-cluster environments in which Caspian interacts with the multi-cluster management platform. Caspian periodically collects information about workloads and clusters, then decides on when and where workloads are executed. Such scheduling and placement decisions are communicated to the multi-cluster manager, which in turn applies those decisions and handles the execution of workloads over the multiple clusters.</i>                                      |
| 해석 | 우리는 <b>Caspian</b> 이 멀티클러스터 관리 플랫폼과 상호작용하는 멀티클러스터 환경을 위한 프레임워크를 설계했다. <b>Caspian</b> 은 워크로드와 클러스터에 대한 정보를 주기적으로 수집하고, 워크로드가 언제, 어디서 실행될지를 결정한다. 이러한 스케줄링·배치 결정은 멀티클러스터 매니저에 전달되며, 매니저는 그 결정을 적용하고 여러 클러스터에 걸쳐 워크로드 실행을 처리한다.                                                                                                                                                                                                                                                                             |
| 설명 | 우리가 만든 시스템의 동작 흐름: 정보 수집 → 최적 계획 수립 → 명령 전달 → 실행                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 원문 | <i>Employing a running implementation in multiple Kubernetes clusters, we evaluated Caspian using real-world carbon intensity data and workloads with varying characteristics. The experimental results showed that Caspian achieves high carbon reduction compared to a baseline scheduler which only considers the QoS of workloads.</i>                                                                                                                                                               |
| 해석 | 여러 쿠버네티스 클러스터에서 실제로 돌아가는 구현을 활용하여, 실세계 탄소집약도 데이터와 다양한 특성의 워크로드로 <b>Caspian</b> 을 평가했다. 실험 결과 <b>Caspian</b> 이 QoS만 고려하는 베이스라인 스케줄러에 비해 높은 탄소 절감을 달성함을 보였다.                                                                                                                                                                                                                                                                                                                                               |
| 설명 | 이론이 아니라 실제로 돌아가는 소프트웨어를 만들고, 실제 탄소 데이터로 실험했다. 현실에서 작동한다!                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 원문 | <i>In the future, we plan to enhance Caspian by: (i) investigating the potential of considering clusters with heterogeneous processing power, (ii) integrating machine learning algorithms to predict the power consumption of workloads, and (iii) extending the experiments by considering real-word workloads.</i>                                                                                                                                                                                    |
| 해석 | 향후 우리는 다음과 같이 <b>Caspian</b> 을 개선할 계획이다: (i) 이기종 처리 성능을 가진 클러스터를 고려할 가능성 탐구, (ii) 워크로드의 전력 소비를 예측하는 머신러닝 알고리즘 통합, (iii) 실제 워크로드를 고려한 실험 확장.                                                                                                                                                                                                                                                                                                                                                              |
| 설명 | (i) 현재 모든 서버가 동일한 GPU 성능을 가진다고 가정 → 실제로는 A100, H100, V100 등 세대가 섞임. 이걸 고려해야 함.<br>(ii) 현재 전력 소비를 GPU 사용률에 선형 비례로 단순화 → ML로 더 정확하게 예측.<br>(iii) 현재 실험은 sleep 명령 기반 가짜 워크로드 → 실제 AI 학습 잡으로 실험 필요.<br>원문: "we plan to enhance Caspian by: (i) investigating the potential of considering clusters with heterogeneous processing power, (ii) integrating machine learning algorithms to predict the power consumption of workloads, and (iii) extending the experiments by considering real-word workloads." |

-----

- [1] B. Acun, B. Lee, F. Kazhamiaka, K. Maeng, U. Gupta, M. Chakkaravarthy, D. Brooks, and C.-J. Wu. Carbon explorer: A holistic framework for designing carbon aware datacenters. In *ASPLOS*, Vol. 2, pp. 118–132, 2023.
- [2] T. Bahreini, A. Tantawi, and A. Youssef. An approximation algorithm for minimizing the cloud carbon footprint through workload scheduling. In *IEEE CLOUD*, pp. 522–531, 2022.
- [3] T. Bahreini, A. Tantawi, and A. Youssef. A carbon-aware workload dispatcher in cloud computing systems. In *IEEE CLOUD*, pp. 212–218, 2023.
- [4] N. Bashir, T. Guo, M. Hajiesmaili, D. Irwin, P. Shenoy, R. Sitaraman, A. Souza, and A. Wierman. Enabling sustainable clouds: The case for virtualizing the energy system. In *ACM SoCC*, pp. 350–358, 2021.
- [5] M. Ehrgott. *Multicriteria optimization*, volume 491. Springer, 2005.
- [6] X. Fan, W.-D. Weber, and L. A. Barroso. Power provisioning for a warehouse-sized computer. *ACM SIGARCH*, 35(2):13–23, 2007.
- [7] H. Ferreboeuf et al. Lean ict: Towards digital sobriety. *The Shift Project*, 6:16–28, 2019.
- [8] Í. Goiri, K. Le, T. D. Nguyen, J. Guitart, J. Torres, and R. Bianchini. Greenhadoop: leveraging green energy in data-processing frameworks. In *EuroSys*, pp. 57–70, 2012.
- [9] O. Grodzevich and O. Romanko. Normalization and other topics in multi-objective optimization. 2006.
- [10] W. A. Hanafy, Q. Liang, N. Bashir, D. Irwin, and P. Shenoy. Carbonscaler: Leveraging cloud workload elasticity for optimizing carbon-efficiency. *SIGMETRICS*, 52(1):49–50, 2024.
- [11] L. Joppa. Made to measure: Sustainability commitment progress and updates. *Microsoft Blog*, July 2021.
- [12] R. Koningstein. We now do more computing where there's cleaner energy (2021). Google Blog.
- [13] E. Masanet, A. Shehabi, N. Lei, S. Smith, and J. Koomey. Recalibrating global data center energy-use estimates. *Science*, 367(6481):984–986, 2020.
- [14] T. Mastelic, A. Oleksiak, H. Claussen, I. Brandic, J.-M. Pierson, and A. V. Vasilakos. Cloud computing: Survey on energy efficiency. *ACM Computing Surveys*, 47(2):1–36, 2014.
- [15] A. Radovanovic, R. Koningstein, I. Schneider, B. Chen, A. Duarte, B. Roy, D. Xiao, M. Haridasan, P. Hung, N. Care, et al. Carbon-aware computing for datacenters. *arXiv:2106.11750*, 2021.
- [16] I. Rocha, C. Göttel, P. Felber, M. Pasin, R. Rouvoy, and V. Schiavoni. Heats: Heterogeneity-and energy-aware task-based scheduling. In *PDP*, pp. 400–405. IEEE, 2019.
- [17] A. Souza, S. Jasoria, B. Chakrabarty, A. Bridgwater, A. Lundberg, F. Skogh, A. Ali-Eldin, D. Irwin, and P. Shenoy. Casper: Carbon-aware scheduling and provisioning for distributed web services. In *IGSC '23*, pp. 67–73, 2024.
- [18] M. Tirmazi, A. Barker, N. Deng, M. E. Haque, Z. G. Qin, S. Hand, M. Harchol-Balter, and J. Wilkes. Borg: the next generation. In *EuroSys*, pp. 1–14, 2020.
- [19] Z. Wang, S. Chen, L. Bai, J. Gao, J. Tao, R. R. Bond, and M. D. Mulvenna. Reinforcement learning based task scheduling for environmentally sustainable federated cloud computing. *Journal of Cloud Computing*, 12(1):1–17, 2023.

서론의 "메타 87% SLO 통계"의 출처. 메타의 탄소인지 데이터센터 설계 프레임워크.

저자들의 이전 논문: 시간 이동(temporal shifting)만 다룸.

저자들의 이전 논문: 공간 배치(spatial placement) +  $Z_1$ 의 NP-hardness 증명 출처.

에너지 시스템 가상화를 통한 지속가능 클라우드 관련 연구.

스칼라화(scalarization) 기법의 이론적 출처.

"전력 소비  $\propto$  자원 사용률" 선형 모델의 근거. 식(1)의 이론적 토대.

"데이터센터 탄소 2.5~3.7%" 수치의 출처.

데이터 처리 프레임워크에서 녹색 에너지 활용 연구.

다목적 최적화에서 정규화 방법들 비교 연구.

워크로드 탄력성을 활용한 탄소 효율 최적화. Caspian(이동/배치)과 상호보완적.

마이크로소프트의 재생에너지 투자 사례.

구글의 탄소인지 컴퓨팅 배포 사례.

"데이터센터 전력 1~2%" 수치의 출처.

클라우드 에너지 효율 서베이. 기존 자원관리 플랫폼의 한계를 보여줌.

구글의 실전 탄소인지 컴퓨팅 배포 논문.

이기중·에너지인지 작업 기반 스케줄링 연구.

부산 웹 서비스의 탄소인지 스케줄링. 이름이 Caspian과 비슷하지만 다른 연구.

"구글 Borg best-effort 잡 비중" 수치의 출처. 배치 작업이 미룰 수 있다는 전제의 근거.

강화학습 기반 탄소인지 페르레이티드 클라우드 스케줄링.